

DURGAM

Digital University Resource for Governance & Academic Management
Request for Proposal — Build Specification (v3)
Sri Sathya Sai Institute of Higher Learning (Deemed to be University)

Version 3.0 • May 2026

Document purpose

This document is the formal specification for **DURGAM**, the integrated University ERP. It supersedes the informal requirements draft and the earlier v1 and v2 specifications. It is written to be read by two audiences in the same sitting:

- the **Claude Code** agent that will write the application, and
- the **University administrator** who will commission, review and accept that work.

Every technical section is preceded by a short *In plain language* explanation. Technical readers may skip those; non-technical readers can rely on them and, where curious, read on into the formal text.

Contents

1	How to Read This Document	4
1.1	For the University Administrator	4
1.2	For Claude Code	4
1.3	What Changed Between v1 and v2	4
1.4	What Changed Between v2 and v3	4
2	Executive Summary	5
2.1	The Project	5
2.2	Engagement Model	6
2.3	Non-Negotiable Constraints	6
2.4	Feasibility for Claude Code	6
3	Glossary & Conventions	6
4	Technology Stack	7
4.1	Development Stack	7
4.2	Production Stack	8
5	System Architecture	10
5.1	Logical View	10
5.2	Cross-Cutting Concerns (mandatory in every module)	10
5.3	Component Diagram (textual)	10

6	Constraints & Non-Functional Requirements	11
6.1	Security	11
6.2	Privacy	12
6.3	Performance	12
6.4	Responsiveness & Accessibility	12
6.5	Auditability	12
7	Roles, Permissions & Access Control	12
7.1	Role Catalog	13
7.2	Permission Model	13
7.3	Role-Based Email Identities	13
8	Canonical Data Model	14
8.1	Identity, Roles, Permissions	14
8.2	Organisational Structure	15
8.3	People	17
8.4	Cross-cutting: Audit, Files, Notifications, Approvals	18
8.5	Calendar & Configuration Anchors	19
8.6	Indexes & Constraints (representative)	20
9	Module Specifications	21
9.1	Authentication	21
9.2	Admin Module	21
9.3	Configuration Module	22
9.4	Audit Module	22
9.5	Approval Request Module	22
9.6	Announcement Module	23
9.7	Faculty Module	23
9.8	Student Module	24
9.9	Research Scholars Module	24
9.10	Department Module	24
9.11	Program, Course Modules	25
9.12	Campus Module	25
9.13	CESRC Module	26
9.14	Attendance Module	27
9.15	Examination Module	27
9.16	Reports Module	27
9.17	Modules Deferred (per source requirements)	27
10	Module Dashboards	28
10.1	Standard Widgets	28
10.2	Per-Role Dashboards	28
10.3	Implementation Notes	29
11	Leave Rules Module	29
11.1	Leave Types Supported	30
11.2	Universal Rules (applied to every leave type)	30
11.3	Casual Leave (CL) — detailed rules	30
11.4	Special Casual Leave (SCL) — detailed rules	30
11.5	Earned Leave (EL) — detailed rules	31
11.6	Half-pay Leave (HPL) and Commuted Leave (CML) — detailed rules	31

11.7 Extraordinary Leave (EOL) — detailed rules	31
11.8 Maternity Leave (ML) — detailed rules	32
11.9 Study Leave (SL) — detailed rules	32
11.10 Sanctioning Authority Matrix	32
11.11 Special Cases	33
11.12 Data Model for Leave	33
11.13 Behavioural Rules (engine)	34
11.14 User Experience	35
11.15 Seed YAML for the Sanctioning Authority Matrix	35
11.16 Acceptance Criteria for the Leave Module	36
12 Implementation Order & Milestone Gates	36
13 Testing Strategy	39
13.1 Levels of Test	39
13.2 Specific Test Requirements per Concern	40
13.3 Coverage Targets	40
13.4 Continuous Integration	41
13.5 Manual Test Plan	41
14 Acceptance Criteria & Quality Bar	41
14.1 Per-Milestone Gate Checklist	41
14.2 Programme-Level Bar	42
15 Theme & Visual Identity	42
15.1 The Single Committed Theme: Puttaparthi Saffron–Indigo–Ivory	42
15.2 Implementation	42
16 Risks & Best-Available Mitigations	42
17 Feasibility for Claude Code	44
17.1 Bottom Line	45
17.2 What Claude Code Will Do Well	45
17.3 What Needs Human Decision & Review	45
17.4 How to Drive Claude Code on This Project	45
18 Appendix A — Seed Data Reference	46
19 Appendix B — Repository Layout	46
20 Appendix C — The Administrator’s One-Page Checklist	47

1 How to Read This Document

In plain language: This document tells the AI coding assistant exactly what to build, in what order, and how the University will check the work at every step. You do not need to be a programmer to use it. The document is organised so that any single module can be approved by the University before the next one begins.

1.1 For the University Administrator

You should read, in order: this section, the *Executive Summary*, the *Glossary*, the *Implementation Order & Milestone Gates*, and *Acceptance Criteria*. Every other section opens with a plain-language explanation that summarises what is being committed.

When work is delivered for review, ask three questions for each milestone:

1. Does the demonstration show every behaviour listed in this document for that milestone?
2. Did all the automated tests pass on the day of delivery? (You should see a green check in the test report.)
3. Has the documentation for that module been updated? (One Markdown file per module under `docs/modules/`.)

If the answer to all three is yes, the milestone gate is passed.

1.2 For Claude Code

Treat this document as the contract. Every module reference (e.g. “§9”) is an exact pointer. Schemas in §8 are canonical. Implementation order in §12 is binding. Tests are part of every deliverable, not an afterthought; the testing requirements in §13 are mandatory at every milestone gate.

1.3 What Changed Between v1 and v2

- **Removed** WhatsApp messaging from scope (delivery via email + in-app only).
- **Removed** in-meeting browser audio recording from scope.
- **Removed** AI-generated minutes-of-meeting from scope.
- **Reduced** themes to a single elegant theme (Puttaparthi Saffron–Indigo–Ivory). The implementation will still expose theme tokens centrally, so additional themes can be added later if desired, but no alternative themes are committed for v1.
- **Added** a fully encoded *Leave Rules* module (§11) drawn directly from the Rules of SSSIHL, 2020 (Section XXVIII).
- **Added** a comprehensive *Testing Strategy* (§13).
- **Added** plain-language explanations throughout for non-technical readers.
- **Resolved** every risk previously marked “open” — either by removal of scope or by committing to a specific best-available open-source approach (§16).

1.4 What Changed Between v2 and v3

v3 closes the coverage gaps identified in the side-by-side comparison of v2 against the original informal requirements, plus a small set of operational refinements. No descopes have been reversed; the changes are additive (or strengthen what was already there).

- **Added** a `School` model and a `school_id` foreign key on `Department`, so the four schools are first-class data objects rather than only role hierarchy (§8, §9).
- **Expanded** the `Program` model with PEOs, POs, PSOs, regulations PDF (with effective-from AY), schemes of instruction, specialisations, and exit levels with recognition types — all NAAC- and NEP-relevant fields that v2 omitted (§8, §9).
- **Promoted** the CESRC (Community Engagement & Social Responsibility Cell) module from deferred to active scope, with a lightweight specification mirroring the `Campus` module shape (§9).
- **Added** a `StudentCategoryCount` AY-scoped table for SC/ST/OBC/EWS/General counts per academic year (§8, §9).
- **Added** the *campus people directory* view (students, faculty, non-teaching staff, sensitive PII excluded) under the `Campus` module (§9).
- **Enumerated** the full waste-disposal field set in the `Campus` module (§9).
- **Added** the mental-well-being club counsellor-validation rule (counselling activities permitted only with currently-configured campus counsellors) under the `Campus Clubs` sub-module (§9).
- **Added** the finance-request budget-heading check to the `Approval Module`: when a department or campus raises a finance request, the system asks whether the heading was claimed in the annual budget (§9).
- **Added** a Central Purchase Committee approval flow for fund-release requests (§9).
- **Expanded** the `Student Module`’s parental feedback channel to a five-bullet behaviour specification matching the depth of other modules (§9).
- **Added** a per-module *Dashboards* specification describing the standard widgets each role’s dashboard exposes (§10).
- **Wired** the `Holiday` table into the `Leave Rules`’ `compute_leave_days` contract; added a YAML-seed example for the `LeaveSanctionAuthorityRule` (§11).
- **Clarified** that development email runs through Mailpit (open-source) from M1 through M19; production SMTP & DMARC are due at M20 (§16).
- **Replaced** the open-ended “sponsor will supply seed data” note with a concrete fallback: synthetic mock data generated by a seed script unblocks development; the M5 bulk-import flow accepts the real data when it arrives, regardless of timing (§16).

2 Executive Summary

In plain language: DURGAM is the University’s own software system. It will hold and run the work of admissions, classes, attendance, examinations, faculty profiles, departmental governance, campus management and audit, in one place. It will be built entirely from free, open-source tools, and it will be built carefully, one module at a time, with an automated test suite that proves each module works before the next one is started.

2.1 The Project

DURGAM — *Digital University Resource for Governance & Academic Management* — is the integrated, in-house ERP platform of Sri Sathya Sai Institute of Higher Learning (Deemed to be University). Its mandate is to consolidate fragmented administrative, academic and governance workflows under a single, secure, role-aware digital ecosystem spanning four campuses (Prasanthi Nilayam, Brindavan, Muddenahalli/Nandigiri, Anantapur), ten academic departments and

the associated centres, committees and support services.

The name draws from the Sanskrit *durgam* (lit. “difficult to access, hard to navigate”). DURGAM is the University’s effort to transform that historical complexity into a unified, accessible, future-ready platform. The name also resonates with Goddess Durga, a symbol of strength, protection and the overcoming of obstacles — the spirit in which the system aspires to serve students, faculty, administrators and governance bodies alike.

2.2 Engagement Model

The system will be implemented by a human engineering lead working with **Claude Code** as the primary coding agent. This RFP defines the scope, architecture, data model, module-implementation order and acceptance criteria with sufficient precision that Claude Code can be tasked module-by-module, with each task bounded, reviewable and verifiable.

2.3 Non-Negotiable Constraints

- Implementation framework: **Reflex** (pure-Python full-stack, Apache 2.0).
- Database: **PostgreSQL** (open source).
- Entire stack must be **free / open-source**; no commercial licences, no per-seat fees, no per-message fees.
- Application must be **production-transition ready** from day one: containerised, configuration-driven, observable, recoverable.
- **Authentication-gated, role-based access control, full audit trail, soft + hard delete** with system-admin-only hard delete, **confirmation on every delete, responsive on phone, tablet, desktop, wide screens**.
- Single elegant theme: Puttaparthi Saffron–Indigo–Ivory, inspired by Sri Sathya Sai Hostel buildings of Puttaparthi.

2.4 Feasibility for Claude Code

A direct feasibility assessment is presented in §17. In short: **the project is achievable with Claude Code, provided the work is decomposed into the milestone sequence in §12 and a human reviewer enforces every milestone gate.**

3 Glossary & Conventions

In plain language: This is the shared vocabulary. Whenever a word in this list appears later in the document, it carries the meaning given here.

Soft delete

A record marked inactive. It is hidden from normal screens but remains in the database; can be restored.

Hard delete

Permanent removal from the database. *Allowed only for the System Admin*, and only after the record is already soft-deleted, and only after explicit confirmation.

Audit trail

A permanent, append-only log of every action that changes data: who did what, when, and what changed.

CRUD

The four basic operations on a record: Create, Read, Update, Delete.

RBAC

Role-Based Access Control. Permissions are granted to roles; users hold one or more roles.

Academic Year (AY)

The University’s academic session, e.g. AY 2025–26. Data scoped to an AY becomes immutable when the AY rolls over.

Module

A vertical slice of functionality (Admin, Library, Hostel, etc.). Each module owns one or more groups of database tables.

Role-based email

An email address attached to an institutional role (e.g. `registrar@...`, `iqac@...`). Distinct from the user’s personal official email; used for outbound system actions executed in that role.

Approval Process

A configurable chain of approvers attached to a request type, with optional uploads in either direction.

Letterhead

A pre-uploaded template image merged into auto-generated reports.

Milestone gate

A formal review point at the end of each milestone. Work cannot proceed past the gate until the gate’s checklist is fully ticked.

Vacation employee

As defined in the Leave Rules: Professors, Associate Professors, Assistant Professors, Instructors — but *not* Directors of campuses.

Non-vacation employee

All employees who are not vacation employees (administrative staff, Directors, etc.).

4 Technology Stack

In plain language: Below are the building blocks the system will be made of. Every item is free and open-source. The “development stack” is what the engineers and Claude Code use while writing the system. The “production stack” is what runs the live system once it is deployed.

4.1 Development Stack

Concern	Tooling
Application framework	Reflex (pure-Python full-stack; compiles UI to React, back-end to FastAPI).
Language	Python ≥ 3.11 (latest stable supported by Reflex at project start).
Package & env management	<code>uv</code> (recommended by Reflex) or <code>poetry</code> ; pinned versions via lockfile.
Database	PostgreSQL ≥ 15 with extensions <code>pgcrypto</code> (column encryption), <code>citext</code> (case-insensitive emails), <code>uuid-oss</code> (UUID generation).
ORM & migrations	SQLModel on top of SQLAlchemy 2.x; Alembic for migrations.
State / cache	Redis (Reflex’s recommended state manager in production).
Background jobs	Celery + Redis broker for heavy or scheduled work; AP-Scheduler for lighter cron-like tasks.
Search	PostgreSQL full-text search (<code>tsvector/tsquery</code>); cross-module search remains within Postgres unless a future need demands otherwise.
Auth	Reflex auth + <code>passlib[bcrypt]</code> for password hashing; signed session tokens; sliding-expiry sessions to support the 7-day idle window.
File storage	Local filesystem in dev; MinIO (S3-compatible, open-source, Apache 2.0) in production — abstracted behind a storage interface.
Email	SMTP via <code>aiosmtp</code> ; rich-text composition in the browser via TipTap or Quill (both MIT-licensed); transactional templates rendered with Jinja2.
DOI / publication metadata	CrossRef public REST API (<code>api.crossref.org</code>); DataCite as fallback. Both free, public, no API key required.
Document generation	<code>python-docx</code> (Word), <code>python-pptx</code> (PowerPoint), WeasyPrint (HTML-to-PDF) and <code>reportlab</code> (programmatically PDF), <code>openpyxl</code> (Excel), <code>pandas</code> (CSV/Excel exports).
Logging	<code>structlog</code> with JSON sinks; rotated via <code>logrotate</code> or container log driver.
Testing	<code>pytest</code> , <code>pytest-asyncio</code> , <code>pytest-cov</code> for coverage, <code>hypothesis</code> for property-based tests, <code>Playwright</code> (Python) for end-to-end browser tests, <code>factory_boy</code> or <code>polyfactory</code> for test fixtures.
Linting / formatting	<code>ruff</code> , <code>black</code> , <code>mypy</code> .
Pre-commit	<code>pre-commit</code> with the above hooks.
Version control	Git. Conventional Commits. Trunk-based development with short-lived feature branches.

4.2 Production Stack

Concern	Tooling
Containerisation	Docker; multi-stage images.

Concern	Tooling
Orchestration	Docker Compose for the single-host deployment; K3s (lightweight Kubernetes) once horizontal scale becomes necessary.
Reverse proxy / TLS	Caddy (auto-HTTPS) or Nginx + Certbot.
Rate limiting / WAF	Caddy/Nginx rules + <code>slowapi</code> at the FastAPI layer.
Object storage	MinIO clustered, with versioning enabled for uploaded letterheads, certificates and other persistent files.
Database HA	PostgreSQL with streaming replication; <code>pgBackRest</code> for point-in-time recovery.
Cache	Redis (sentinel for HA when needed).
Job runners	Celery workers in a separate container set.
Observability	Prometheus + Grafana (metrics); Loki (logs); OpenTelemetry traces to Tempo or Jaeger. All open-source.
Backups	Nightly <code>pg_dump</code> + <code>pgBackRest</code> continuous archive; MinIO bucket replication; quarterly tested restore drill.
Secrets	age-encrypted env files held in a private repository, or HashiCorp Vault OSS. Never plaintext, never in source.
CI/CD	GitHub Actions (or Gitea Actions for self-hosted): build, test, scan, push, deploy.
Static & dependency security	<code>bandit</code> (code), <code>pip-audit</code> (Python deps), <code>trivy</code> (container images).
Application error tracking	Sentry, self-hosted (open-source).
Antivirus on uploads	ClamAV (open-source).

Why no WhatsApp / no in-meeting recording / no AI-generated minutes

These were considered in v1 but removed in v2 by sponsor decision. Reasons:

- **WhatsApp:** the official Meta Cloud API is not free of cost beyond a small free tier; pure-OSS alternatives (Web-protocol bridges) are unstable and put the institution's WhatsApp account at risk of suspension. The system therefore commits to email and in-app notifications only, both of which are free, reliable, and open.
- **In-meeting audio recording:** not a stable browser-only feature in a Reflex application; the institutional and privacy implications are substantial; meeting recording, where required, can be done with off-the-shelf tools and the recording uploaded to the system as a file attachment.
- **AI-generated minutes:** self-hosted open-source LLMs cannot be relied upon to produce minutes of the quality required for a Board of Studies or governance meeting. Meeting minutes will be drafted by humans using the in-app rich-text editor and uploaded to the meeting record.

The system will continue to support file uploads of recordings and minutes as attachments to meeting records.

5 System Architecture

In plain language: The system is built as one well-organised codebase (a “modular monolith”). Inside, it is divided into clean layers — screens at the top, business logic in the middle, the database at the bottom — with shared services for security, notifications and audit. Built this way, individual modules can later be split into separate services if growth demands, without rewriting them.

5.1 Logical View

DURGAM is a **modular monolith** at delivery, designed so that hot modules (Audit, Examination, Approval) can be extracted into independent services later without rewriting integration points.

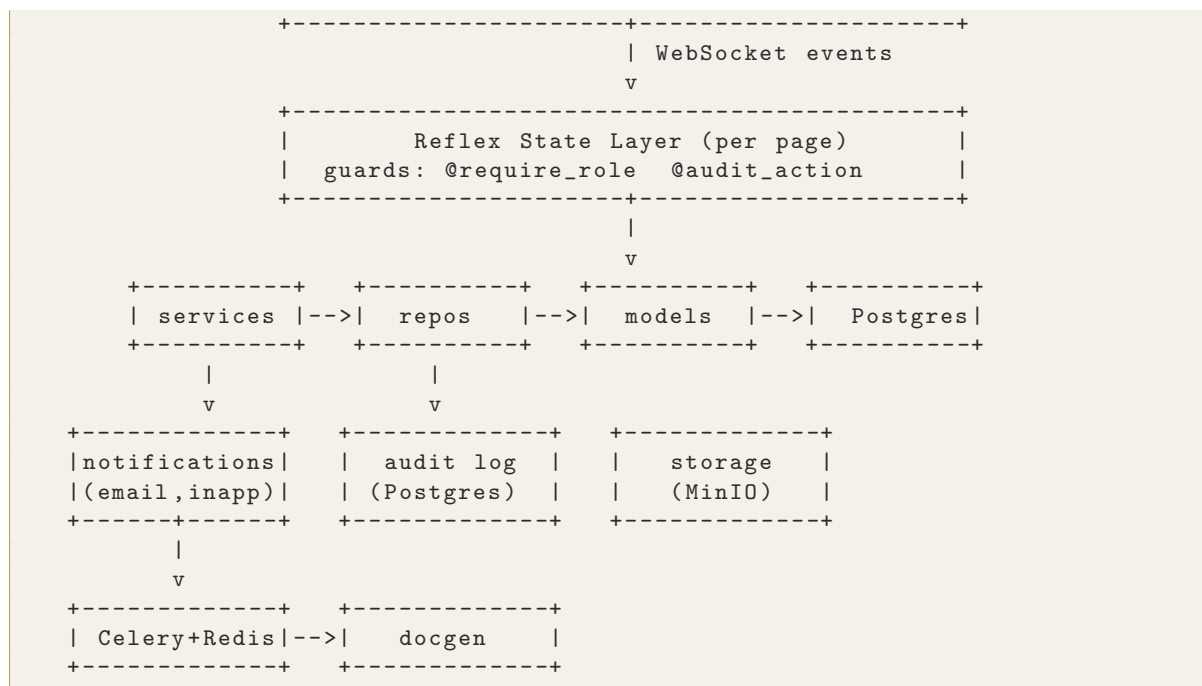
- **Presentation layer:** Reflex pages & components — one Reflex app, multi-page, role-gated routing.
- **Application layer:** Reflex **State** classes (per page) delegate to *service* classes that own business rules; states never talk to the ORM directly.
- **Service layer:** Pure-Python services (**services/**); one per domain.
- **Repository layer:** **repositories/** encapsulate SQLAlchemy queries; services consume repositories.
- **Domain models:** **models/** — SQLAlchemy classes; one file per aggregate.
- **Cross-cutting:** **audit/**, **auth/**, **permissions/**, **notifications/**, **storage/**, **docgen/**, **integrations/**.
- **Background:** **jobs/** — Celery tasks (email send, calendar reminders, scheduled report generation, AY-rollover lock).

5.2 Cross-Cutting Concerns (mandatory in every module)

1. **Permission decorator** on every state event handler and every page.
2. **Audit decorator** that records actor, action, target, before/after diff, request id.
3. **Soft-delete** default; hard delete is a separate, system-admin-only endpoint.
4. **Confirmation dialog** on every destructive action (delete, cancel, withdraw, finalize).
5. **Notification dispatcher** routes to email and in-app based on event type and user preference.
6. **Responsive layouts:** mobile-first; layouts validated at 360 px, 768 px, 1280 px, 1920 px.
7. **Theme provider:** a single **ThemeContext** consumed by all components; tokens live in one place.

5.3 Component Diagram (textual)





Listing 1: High-level component layout.

6 Constraints & Non-Functional Requirements

In plain language: This section sets the bar for security, privacy, speed and accessibility. These are not features — they are properties every part of the system must satisfy.

6.1 Security

- Passwords hashed with bcrypt (cost factor ≥ 12); never stored in plaintext or any reversible form.
- Password policy: minimum 12 characters; at least one uppercase, one lowercase, one digit, one symbol; deny common-password list (top 10 000); deny passwords containing the user's email local-part or full name.
- Forced password change on first login; subsequent rotation *not* mandated (per stakeholder direction in source requirements).
- Session: signed, HTTP-only, Secure, SameSite=Lax cookie; 7-day sliding inactivity expiry as specified.
- All HTTP traffic over TLS in production; HSTS enabled.
- CSRF protection on every state-changing request.
- Per-user and per-IP rate limiting on login and password-reset endpoints.
- File uploads: type & size validation server-side; antivirus scan via ClamAV before persistence; stored under non-guessable UUID keys.

- Sensitive PII (Aadhaar, PAN) encrypted at column level using **pgcrypto**; access tightly scoped to Registrar / IQAC / System Admin.
- Security baseline: OWASP ASVS Level 2.

6.2 Privacy

- Audit log captures *action* and *target id*, not full PII payloads, except where the diff is itself the action's substance (e.g. profile edits).
- Soft-deleted users cannot log in but their historical audit entries remain.
- Hard delete cascades according to a documented retention matrix; legal-hold flag overrides hard delete.

6.3 Performance

- Median page load ≤ 1.5 s on a 4G connection from within India.
- Dashboard widgets render with skeleton state within 250 ms; data hydrates progressively.
- Long reports generated asynchronously; user receives in-app + email link when ready.

6.4 Responsiveness & Accessibility

- Mobile-first layouts; navbar collapses to drawer below 768 px.
- WCAG 2.1 AA compliance: colour contrast, full keyboard navigation, ARIA labels, visible focus indicators.
- Theme contrast validated by axe-core in CI on every page.

6.5 Auditability

- Every state-changing action (including login, logout, failed login) produces an audit row.
- Audit table is append-only; no UPDATE/DELETE granted to the application database role; only System Admin can read.
- Searchable by actor, role, target table, target id, action type, date range; downloadable as CSV / Excel / PDF / DOCX.

7 Roles, Permissions & Access Control

In plain language: Every person in the system has one or more “roles” (Faculty, HoD, Registrar, etc.). Each role can do certain things and see certain pages; roles can also be limited to a particular department or campus. The same person can hold several roles at the same time — they get the combined ability of all their roles.

7.1 Role Catalog

Roles are seeded from a single source-of-truth YAML file at first boot. Every role implies a set of permissions; a user's effective permission set is the **union** of permissions across all roles assigned. Every authenticated user implicitly carries the **basic_user** role.

Levels:

- **University:** VC, VC Office, Registrar, Deputy Registrar, Registrar Office, CoE, Deputy CoE, CoE Office, HR Head, HR Office.
- **Campus:** Director, Deputy Director, Director Office.
- **School:** Dean of Sciences, Dean of Humanities & Social Sciences, Dean of Languages & Literature, Dean of Management & Commerce, and the corresponding Dean Offices.
- **Department:** HoD, AHoD, HoD Office.
- **Class:** Class Teacher.
- **Centres for Excellence:** Centre Chairperson, Centre Coordinator, Centre Office.
- **CRIF:** CRIF Convener, CRIF Office.
- **CESRC:** CESRC Chairperson, CESRC Coordinator, CESRC Office.
- **Statutory committees:** Committee Chairperson/President, Committee Office.
- **IQAC:** IQAC Chairperson, IQAC Coordinator, IQAC Deputy Coordinator, IQAC Office.
- **Library:** Librarian, Campus Librarian, Librarian Office, Campus Librarian Office.
- **Hostel:** Campus Hostel Warden, Deputy Warden, Hostel Office.
- **Placement:** Placement Officer, Department Placement Coordinator, Placement Office.
- **LMS:** LMS Admin, Department LMS Coordinator, LMS Office.
- **Other:** Dean of Academic Affairs, Dean of Student Welfare, Dean of Innovation & Entrepreneurship, Faculty, Student, Guest, Basic User, System Admin.

7.2 Permission Model

Permissions are encoded as (resource, action, scope) triples:

- **resource** — e.g. `course`, `faculty`, `announcement`, `audit_log`, `leave_request`.
- **action** — one of `create`, `read`, `update`, `soft_delete`, `hard_delete`, `approve`, `export`, `configure`.
- **scope** — one of `global`, `university`, `campus`, `school`, `department`, `class`, `centre`, `committee`, `self`.

The permission check is centralised in `auth/permissions.py::can(user, action, resource, target=None)`. It evaluates scope against the target (e.g. a department-scoped HoD permission resolves true only for targets within that HoD's department). System Admin's permission set is ('*', '*', 'global').

7.3 Role-Based Email Identities

A faculty member may simultaneously be Faculty, IQAC Coordinator and Dean of Sciences. Each role has an associated official email (configured in the Configuration module by Registrar / Deputy Registrar / Registrar Office / System Admin). When the user performs an action whose “official voice” is the role (e.g. Dean issuing a notice), outbound mail uses the role-based From address. When acting in their personal-faculty capacity, outbound mail uses the personal faculty address. The active role context is always shown in the UI and is selectable when more than one applies.

8 Canonical Data Model

In plain language: The shape of the data the system stores. Each block below is a database table. These are written for engineers; the administrator does not need to read the code, but should know that this section is the authoritative reference for what is stored about whom.

Standard audit columns — `created_at`, `updated_at`, `created_by`, `updated_by`, `is_deleted`, `deleted_at`, `deleted_by` — apply to *every* table and are factored into a `TimestampedSoftDelete` mixin; they are omitted from each model's listing for brevity.

8.1 Identity, Roles, Permissions

```
class User(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    username: str = Field(unique=True, index=True)          # local-part of
        email
    email: EmailStr = Field(unique=True, index=True)
    password_hash: str
    is_active: bool = True                                  # inactive => cannot
        log in
    must_change_password: bool = True                       # first-login flag
    last_login_at: datetime | None = None
    profile_completed: bool = False                         # for faculty/
        student
    aadhaar_enc: bytes | None = None                        # pgcrypto column
    pan_enc: bytes | None = None                           # pgcrypto column

class Role(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True)                          # 'HOD', 'IQAC_COORD
        ', ...
    name: str
    level: str                                              # 'university'/'
        campus' / ...
    description: str | None = None

class Permission(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    resource: str
    action: str                                             # crud + approve/
        export / ...
    scope: str                                             # global / university
        / ... / self

class RolePermission(SQLModel, table=True):
    role_id: UUID = Field(foreign_key='role.id', primary_key=True)
    permission_id: UUID = Field(foreign_key='permission.id', primary_key=True)

class UserRole(TimestampedSoftDelete, table=True):
    user_id: UUID = Field(foreign_key='user.id', primary_key=True)
    role_id: UUID = Field(foreign_key='role.id', primary_key=True)
    scope_type: str | None = None                          # 'department' / '
        campus' / ...
    scope_id: UUID | None = None
```

Listing 2: Identity and access models.

8.2 Organisational Structure

```

class Campus(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # PSN | BRN | NDG |
    ATP
    name: str
    address: str | None = None

class School(TimestampedSoftDelete, table=True):
    """The four academic schools, each headed by a Dean."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # SCI | HSS | LL |
    MC
    name: str
    dean_role_code: str # 'DEAN_SCI', '
    DEAN_HSS', ...

class Department(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # DBIO, DCHEM, DMACS
    , ...
    name: str
    main_campus_id: UUID = Field(foreign_key='campus.id') # where HoD sits
    school_id: UUID = Field(foreign_key='school.id', index=True)

class DepartmentCampus(SQLModel, table=True):
    department_id: UUID = Field(foreign_key='department.id', primary_key=True)
    campus_id: UUID = Field(foreign_key='campus.id', primary_key=True)
    has_ahod: bool = False

class SubDepartment(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # SDPHIL, SDENG, ...
    name: str
    parent_department_id: UUID = Field(foreign_key='department.id')

class CentreOfExcellence(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # CMB, CSSS, CADS,
    CSD
    name: str
    campus_id: UUID = Field(foreign_key='campus.id')

class Program(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True)
    name: str
    regulator: str # UGC/AICTE/NCTE
    level: str # UG/PG/Professional
    |PhD/Diploma
    is_multidisciplinary: bool = False
    delivery_mode: str # online/in_person
    is_odl: bool = False
    is_joint_degree: bool = False
    joint_partner_indian: str | None = None
    is_twinning: bool = False
    twinning_partner_foreign: str | None = None
    start_year: int
    close_year: int | None = None
    owner_department_id: UUID = Field(foreign_key='department.id')
    has_multiple_exit_levels: bool = False # NEP-relevant

```

```

class ProgramDepartment(SQLModel, table=True):
    program_id: UUID = Field(foreign_key='program.id', primary_key=True)
    department_id: UUID = Field(foreign_key='department.id', primary_key=True)

class ProgramOutcome(TimestampedSoftDelete, table=True):
    """A PEO, PO or PSO of a program. Type discriminates."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    program_id: UUID = Field(foreign_key='program.id', index=True)
    outcome_type: str # 'PEO' | 'PO' | 'PSO'
    sequence: int # ordering within type
    statement: str

class ProgramRegulation(TimestampedSoftDelete, table=True):
    """Regulation document attached to a program, AY-effective."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    program_id: UUID = Field(foreign_key='program.id', index=True)
    effective_from_ay_id: UUID = Field(foreign_key='academicyear.id')
    file_id: UUID = Field(foreign_key='fileasset.id') # PDF, <= 5 MB
    notes: str | None = None

class ProgramSchemeOfInstruction(TimestampedSoftDelete, table=True):
    """How many semesters; which courses in each semester for an AY."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    program_id: UUID = Field(foreign_key='program.id', index=True)
    effective_from_ay_id: UUID = Field(foreign_key='academicyear.id')
    total_semesters: int

class ProgramSchemeCourse(SQLModel, table=True):
    """The course list per semester within a scheme."""
    scheme_id: UUID = Field(foreign_key='programschemeofinstruction.id',
        primary_key=True)
    course_id: UUID = Field(foreign_key='course.id', primary_key=True)
    semester_number: int
    is_elective: bool = False
    elective_basket: str | None = None # group label if elective

class ProgramSpecialisation(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    program_id: UUID = Field(foreign_key='program.id', index=True)
    code: str
    name: str
    description: str | None = None

class ProgramExitLevel(TimestampedSoftDelete, table=True):
    """For multi-exit programs (NEP). Each row is one valid exit point."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    program_id: UUID = Field(foreign_key='program.id', index=True)
    after_semester: int # exit allowed after this sem
    recognition: str # certificate / diploma / degree / honors / honors_research
    description: str | None = None

class Course(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True)
    name: str
    program_id: UUID = Field(foreign_key='program.id')

```

```

credits: int
lecture: int; tutorial: int; practical: int
total_periods: int
evaluation: str # I / E / IE
course_type: str # DSC/DSE/MDC/AEC/
                SEC/VAC/...
delivery_mode: str # inhouse / MOOC
mooc_agency: str | None = None
has_iks: bool = False
is_skill_based: bool = False
is_value_education: bool = False
revised_year: int | None = None

```

Listing 3: Campuses, departments, programs, courses.

8.3 People

```

class Faculty(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    user_id: UUID = Field(foreign_key='user.id', unique=True)
    employee_id: str = Field(unique=True)
    title: str # Dr./Mr./Mrs./Ms.
    first_name: str; middle_name: str | None = None; last_name: str
    designation: str # AsstProf L10/11/12, Assoc, Prof, Sr Prof,
                    Instructor
    is_vacation_employee: bool = True # Per Leave Rules: True for academic
                                     ranks
    department_id: UUID = Field(foreign_key='department.id')
    campus_id: UUID = Field(foreign_key='campus.id')
    joining_date: date
    phone: str; whatsapp: str | None = None; alt_phone: str | None = None
    alt_email: EmailStr | None = None
    photo_path: str | None = None
    emergency_contact_name: str; emergency_contact_relation: str
    emergency_contact_phone: str
    is_phd: bool = False
    phd_thesis_title: str | None = None
    phd_registration_number: str | None = None
    phd_awarding_institution: str | None = None
    phd_year: int | None = None
    orcid: str | None = None; linkedin: str | None = None
    google_scholar: str | None = None; researchgate: str | None = None
    gender: str # required for
               maternity leave eligibility

class Student(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    user_id: UUID = Field(foreign_key='user.id', unique=True)
    roll_number: str = Field(unique=True)
    full_name: str
    year_of_admission: int
    father_name: str | None = None
    mother_name: str | None = None
    guardian_name: str | None = None
    student_phone: str
    parent_phone: str
    parent_email: EmailStr # for parental
                           feedback channel
    emergency_contact_name: str; emergency_contact_relation: str
    emergency_contact_phone: str
    abc_id: str = Field(unique=True)

```

```

program_primary_id: UUID = Field(foreign_key='program.id')
program_secondary_id: UUID | None = Field(foreign_key='program.id')
department_id: UUID = Field(foreign_key='department.id')
campus_id: UUID = Field(foreign_key='campus.id')
class_label: str # IUG/IIUG/IIIUG/IVUG/IPG/IIPG/IPRO/.../IDip/
IIDip
mentor_faculty_id: UUID | None = Field(foreign_key='faculty.id')
category: str | None = None # SC/ST/OBC/EWS/GEN

class ResearchScholar(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    user_id: UUID = Field(foreign_key='user.id', unique=True)
    roll_number: str = Field(unique=True)
    full_name: str
    program_id: UUID = Field(foreign_key='program.id') # PhD program
    department_id: UUID = Field(foreign_key='department.id')
    campus_id: UUID = Field(foreign_key='campus.id')
    funding_type: str # institute/govt/
    project
    funding_org: str | None = None
    project_id: UUID | None = None
    is_jrf: bool | None = None

```

Listing 4: Faculty, students, research scholars.

8.4 Cross-cutting: Audit, Files, Notifications, Approvals

```

class AuditLog(SQLModel, table=True):
    """Append-only. Application user has INSERT/SELECT only."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    occurred_at: datetime = Field(default_factory=datetime.utcnow, index=True)
    actor_user_id: UUID | None = Field(foreign_key='user.id', index=True)
    actor_role_code: str | None = None
    action: str
    resource: str
    resource_id: UUID | None = None
    request_id: str | None = None
    ip: str | None = None
    user_agent: str | None = None
    diff_json: dict | None = Field(default=None, sa_column=Column(JSONB))

class FileAsset(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    storage_key: str
    original_name: str
    mime_type: str
    size_bytes: int
    sha256: str = Field(index=True)
    owner_user_id: UUID = Field(foreign_key='user.id')
    purpose: str # 'letterhead'/'
    certificate'/'...
    metadata_json: dict | None = Field(default=None, sa_column=Column(JSONB))

class Notification(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    recipient_user_id: UUID = Field(foreign_key='user.id', index=True)
    channel: str # 'in_app'/'email'
    subject: str | None = None
    body_html: str | None = None
    body_text: str | None = None
    payload_json: dict | None = Field(default=None, sa_column=Column(JSONB))

```

```

sent_at: datetime | None = None
read_at: datetime | None = None
delivery_status: str = 'pending' # pending/sent/
delivered/failed

class ApprovalProcess(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # e.g. 'VAC', '
        LEAVE_CL'
    title: str
    requestor_role_codes: list[str] = Field(sa_column=Column(JSONB))
    channel_role_codes: list[str] = Field(sa_column=Column(JSONB)) # ordered
    requires_upward_attachments: bool = False
    requires_downward_attachments: bool = False
    max_upward_attachments: int = 0
    max_downward_attachments: int = 0
    max_attachment_mb: int = 5
    is_finance: bool = False # triggers budget-
        heading prompt

class ApprovalRequest(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    process_id: UUID = Field(foreign_key='approvalprocess.id')
    requestor_user_id: UUID = Field(foreign_key='user.id')
    title: str
    payload_json: dict = Field(sa_column=Column(JSONB))
    state: str = 'submitted' # submitted/in_review/approved/rejected/
        withdrawn/cancelled
    current_stage: int = 0
    decided_at: datetime | None = None

class ApprovalStep(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    request_id: UUID = Field(foreign_key='approvalrequest.id', index=True)
    stage: int
    approver_role_code: str
    approver_user_id: UUID | None = Field(foreign_key='user.id')
    decision: str | None = None # approved/rejected
        /forwarded
    comment: str | None = None
    decided_at: datetime | None = None

```

Listing 5: Cross-cutting tables.

8.5 Calendar & Configuration Anchors

```

class AcademicYear(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    code: str = Field(unique=True) # '2025-26'
    starts_on: date; ends_on: date
    is_locked: bool = False # set true on
        rollover

class CalendarEntry(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    academic_year_id: UUID = Field(foreign_key='academicyear.id', index=True)
    title: str
    entry_type: str # 'sem_begin'/'sem_end'/'holiday'/'cie'/'meeting'
        '/...'
    starts_at: datetime; ends_at: datetime
    owner_user_id: UUID = Field(foreign_key='user.id')

```

```

    owner_role_code: str
    scope_type: str | None = None
    scope_id: UUID | None = None
    notes: str | None = None

class Holiday(TimestampedSoftDelete, table=True):
    """Declared holidays of the Institute. Drives leave-day computation."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    academic_year_id: UUID = Field(foreign_key='academicyear.id', index=True)
    holiday_date: date = Field(index=True)
    name: str

class RoleEmail(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    role_code: str
    scope_type: str | None = None
    scope_id: UUID | None = None
    email: EmailStr

class LetterheadAsset(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    role_code: str
    scope_type: str | None = None
    scope_id: UUID | None = None
    file_id: UUID = Field(foreign_key='fileasset.id')

class StudentCategoryCount(TimestampedSoftDelete, table=True):
    """SC/ST/OBC/EWS/General student counts per academic year.
    Maintained by Registrar / Registrar Office; AY-scoped, immutable on
    rollover.
    Visible to all roles except Student and Guest."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    academic_year_id: UUID = Field(foreign_key='academicyear.id', index=True)
    sc_count: int = 0
    st_count: int = 0
    obc_count: int = 0
    ews_count: int = 0
    general_count: int = 0
    notes: str | None = None

```

Listing 6: Academic year, calendar, configuration anchors.

8.6 Indexes & Constraints (representative)

- Unique on (role_code, scope_type, scope_id) in role_email.
- Composite index on auditlog(actor_user_id, occurred_at) and auditlog(resource, resource_id).
- Functional index lower(email) on user for case-insensitive lookup.
- Partial unique index on user(email) WHERE is_deleted = false.
- GIN index on auditlog.diff_json for filtered searches.
- Index on department.school_id; index on programoutcome.program_id; index on programschemeofinstr index on programexitlevel.program_id.
- Unique on (academic_year_id) in studentcategorycount; only one row per AY.
- Unique on (holiday_date, academic_year_id) in holiday.
- All FKs declared ON DELETE RESTRICT except where lifecycle requires cascade (e.g. ApprovalStep cascades from ApprovalRequest; ProgramSchemeCourse cascades from ProgramSchemeOfInstruction).

Schema scope

The above is the foundational core. Module-specific tables (admissions, library, hostel, finance, placement, etc.) are introduced in their respective module specifications and follow the same conventions: UUID primary keys, soft-delete + audit mixin, AY scoping where applicable, and immutability on AY rollover where the source requirements demand it.

9 Module Specifications

In plain language: Each module is a chapter of the system. For every module below, you'll see who is allowed to use it, what it does, and how we'll know it works (the acceptance criteria). The Leave module is described in full in its own section because the rules are detailed.

For brevity, each module below uses a uniform shape: *Purpose* / *Owners* / *Key entities* / *Behaviours* / *Acceptance criteria*.

9.1 Authentication

Purpose: Gate the system; enforce login, password rules, sessions.

Owners: all users (login); System Admin (lifecycle).

Key entities: User, Session, PasswordResetToken.

Behaviours:

- Login by username (= local-part of registered email) + password.
- First-login forces password change and (for Faculty/Student) profile completion.
- Forgot-password: email-link with one-time signed token, 30-min expiry; rate-limited.
- Logout invalidates the session token server-side.
- Closing the browser without explicit logout retains session up to 7 idle days; on the 8th day session auto-expires.
- Inactive users cannot log in (clear, generic error to avoid user-enumeration).
- Every authentication event written to **AuditLog**.

Acceptance criteria: Playwright suite covers all positive and negative paths; no path bypasses the permission decorator; rate limiter blocks brute-force attempts in tests.

9.2 Admin Module

Purpose: System Admin's home — manage users, roles, permissions.

Owners: System Admin only.

Key behaviours:

- Create / list / search / edit / soft-delete / hard-delete users *with role(s) attached at creation*.
- User creation auto-generates a strong password (per policy), sets `must_change_password=true`, and emails the user with credentials and a first-login prompt.
- Edit and delete actions trigger an email to the affected user with a change summary.
- Manage roles: create, edit, delete; assign (**resource**, **action**, **scope**) permissions.
- Every user implicitly has `basic_user`; basic users see only the home page.

9.3 Configuration Module

Purpose: The single seat of all institutional “slow-changing” state.

Owners: System Admin, Registrar / Deputy Registrar / Registrar Office, plus role-scoped owners (HoD for dept vision, Director for campus assets, IQAC for templates, etc.) as listed in the source requirements.

Key entities: Campus, School, Department, SubDepartment, CentreOfExcellence, Program (basic + ProgramOutcome, ProgramRegulation, ProgramSchemeOfInstruction, ProgramSchemeCourse, ProgramSpecialisation, ProgramExitLevel), Course (basic info only here), Faculty/Student (basic info only here), AcademicYear, CalendarEntry, Holiday, RoleEmail, LetterheadAsset, ApprovalProcess, ClassTimingsConfig, MentalHealthCounsellor, FacultyMentorAssignment, ClassTeacherAssignment, NonOwnedCourse, UGTimetable, TemplateAsset (BoS, MoM, VAC certificate), StudentCategoryCount.

Behaviours of note:

- Bulk-add by CSV/Excel import for users, faculty, students, courses, programs.
- Schools are seeded once; departments must declare which school they belong to. The Dean role for each school is bound to the school via `dean_role_code`.
- Academic-year-scoped configurations become immutable on AY rollover (a nightly job sets `academic_year.is_locked=true` and the UI denies edits on locked years).
- Calendar collaboration follows the precedence chain: Registrar locks the master calendar → IQAC adds activities → all other roles fill in their slots, each editing only their own entries while seeing others’ read-only.
- Calendar exports: CSV / Excel / PDF / DOCX.
- Letterheads and templates (BoS, MoM, VAC) are never directly visible to other roles; used internally to compose generated documents.
- Mental-health counsellor roster downloadable as DOCX merged into Director’s letterhead; AY-scoped, immutable on rollover.
- Class teacher / class coordinator assignments auto-flow into faculty workload.
- Timetable for first- and second-year UG courses configured by Director’s office; auto-projected into department timetables.
- **Student category counts (SC / ST / OBC / EWS / General)** per AY managed by Registrar / Registrar Office. Visible to all roles *except* Student and Guest. AY-scoped; immutable on rollover.

9.4 Audit Module

Owners: System Admin only (full read & export). *No other role has even view access.*

Behaviours: fast filter & search across all audit columns; download as CSV / Excel / PDF / DOCX with applied filters; charts of activity volume per resource.

9.5 Approval Request Module

Purpose: The generic engine that powers every multi-stage request in the system.

Owners: requestors (any role with permission); approvers per the configured channel.

Behaviours:

- Generic configurable runtime atop the `ApprovalProcess / ApprovalRequest / ApprovalStep` schema.
- Supports *cancellation* (by requestor before any approval), *withdrawal* (by requestor at any stage), *extension* (by requestor only on time-bound requests), *re-submission* (when configured to allow).

- Attachment caps in both directions per the configured process.
- In-app + email notifications at each stage transition.
- Every state change audited.
- **Finance request budget-heading check.** For approval processes flagged as `is_finance=true` (annual budget, event finance, emergency finance, fund release, etc.), the submission form asks the requestor to declare whether the heading was claimed in the annual budget. The answer (yes / no / not applicable) is stored on the request `payload.json` and is visible to every approver in the channel. Approvers may use this signal but the engine does not block on it.
- **Central Purchase Committee (CPC) flow for fund release.** Configured as a dedicated `ApprovalProcess` (code `CPC_FUND_RELEASE`). Requestors are Faculty (Principal Investigators on sanctioned projects) or HoDs; the channel is HoD/AHoD → Registrar Office → Finance Officer (recommend) → CPC Chairperson → VC. Required attachments: sanction order, utilisation certificate (where applicable). The CPC's decision is the final stage; on approval, the Finance team picks up the approved request manually (Finance module is deferred).
- **Leave approval** is a specialised sub-flow detailed in §11.

9.6 Announcement Module

- Composers: VC, VC Office, Registrar, Registrar Office, CoE, Directors and Director Offices, IQAC Coordinator, CESRC Coordinator, CoE Coordinators, Placement Officer, all Deans, Finance Officer, HR Head, HoDs.
- Display priority on dashboards (top to bottom): VC, VC Office, Registrar, Registrar Office, HR Head, IQAC Coordinator, Deans, Director, Director Office, CoE, Finance Officer, Placement Officer, HoD, CESRC Coordinator, CoE Coordinator. *Very important* announcements override priority. Importance is retained for two working days (Sundays + holidays excluded), then re-sorted by recency. Newer “very important” announcements always rise to the top within the two-day window.
- Targeted to specific role/scope sets; up to 5 attachments, each ≤ 2 MB.
- Auto-announcements wired in from Meetings, Clubs, VAC approvals, etc.

9.7 Faculty Module

Owners: the faculty themselves manage their own record. Registrar / Registrar Office / IQAC have full read access including sensitive fields and uploaded documents.

Submodels: `FacultyEducation`, `FacultyExperience`, `FacultyExpertise`, `FacultyDocument`, `FacultyJournalPublication`, `FacultyBookChapterPublication`, `FacultyConferencePublication`, `FacultyBook`, `FacultyPatent`, `FacultyOtherIPR`, `FacultyAward`, `FacultyExternalCollaboration`, `FacultyResearchProject`, `FacultyConsultancy`, `FacultyEContent`, `FacultyPhDGuidance`, `FacultyResearch`, `FacultyWorkload`, `FacultyRequest` (parametric: leave, FDP, conference, invited talk, membership, WFH, field/industry visit, APC, travel for award/recognition/external duty, in-house research funding, external grant proposal submission, fund release).

Notable behaviours:

- DOI-driven auto-fill for journal/chapter/conference publications via CrossRef + DataCite; only when the DOI is unavailable is manual entry permitted, except *impact factor* and *quartile* fields, which remain “Not Available” rather than allowing manual entry — an integrity guard.
- Cross-tagging of co-authors / co-PIs / co-guides / co-inventors / joint awardees from the institute; saving on one record auto-creates the mirror record on the tagged faculty's page.
- Workload aggregates: courses (from allocation), MDC/awareness courses (from campus of-

fice), class-teacher load (from configuration), VAC approvals.

9.8 Student Module

Students manage their own record. Class label drives lifecycle: Examination module's progression decision (*progresses, stays, to join, discontinued, passed out*) auto-updates class or deactivates the user. SWAYAM/MOOC course log captured with course-mapping and certificate upload. Mentor relationship is read-only and reflects what was seeded by the Configuration module.

Parental feedback channel. A dedicated sub-module captures structured feedback to parents:

- **Composers and timing.** Class teacher and mentor compose feedback. Feedback can be initiated at any time (ad-hoc) or as part of a configured periodic cycle (e.g. end of each CIE block, end of semester). The composer cannot send to a parent of a student outside their class / mentee list.
- **Structured fields.** A single feedback record carries: attendance summary (auto-pulled from Attendance Module — days held, days absent, percentage per course), CIE performance summary (auto-pulled from latest CIE results), conduct notes (free text, max 1000 characters), club / co-curricular participation (auto-pulled from Clubs sub-module), and a closing note from the composer (free text). Auto-pulled fields are read-only at compose time.
- **Review and dispatch.** The composer previews the rendered feedback (a single readable page); dispatch is a separate, audited button. On dispatch, an email is sent to the registered `parent_email` address; an in-app notification is posted to the student's own dashboard so the student is not blindsided; a copy is archived on the student record visible to the student, the class teacher, the mentor, the HoD and the Registrar Office.
- **Editing and recall.** Once dispatched, a feedback record is immutable. If the composer notices an error, they may issue a *correction* feedback (a new record that references the previous record and clearly marks itself as a correction). The original is never altered.
- **Privacy and audit.** The audit log records who composed, who reviewed, who dispatched, and the timestamp of each action. Soft-delete is allowed (with confirmation) and restores the record to draft state; hard-delete is System Admin only.

9.9 Research Scholars Module

Mirror of Faculty's research-related sub-models (publications, projects, awards, etc.) with cross-tagging that reflects to guide / co-guide records. Linked to a PhD program.

9.10 Department Module

Owners: HoD / AHoD / HoD Office (manage); rest view (with finer scoping for some artefacts).
Behaviours:

- Course allocation per semester: pulls programs → courses, faculty pool (own dept, other campuses, scholars, visiting/adjunct/guest with valid availability dates), assigns faculty (or routes course to MOOC). Saving propagates into faculty workload.
- Timetable scheduling: post-allocation; conflict detection across faculty, rooms, classes; first/second-UG slots are read-only mirrors from the campus timetable. DOCX/PDF download.
- BoS meetings: schedule → announce → attendees confirm; minutes are uploaded by the HoD/AHoD/HoD Office as a file (DOCX or PDF) attached to the meeting record; all attachments use the BoS template configured by IQAC.
- Workload export: per-faculty merged DOCX/PDF; Registrar can export across campuses.

- Department events, value-added courses (with the configured VAC approval process: Faculty → AHoD/HoD → Dean of Academic Affairs), CIE scheduling, MoUs (with cross-tag for joint MoUs), approval-driven requests.

9.11 Program, Course Modules

Owners: Dean of Academic Affairs (or office); HoD/AHoD/HoD Office of the owning department (and of any associated departments for jointly-run programs); HoD-designated faculty (revocable any time).

Course module behaviours: Course detail UI exposes COs with Bloom levels, unitwise syllabus (per-unit period count), essential / reference reading, assessment scheme, assessment-tool × CO matrix, CO–PO mapping, MOOC routing fields, IKS / skill / value education flags, and revision year.

Program module behaviours (v3):

- **PEOs / POs / PSOs.** Managed as ordered lists per program via `ProgramOutcome` (type-discriminated). For programs run jointly by multiple departments, all owning HoDs / AHoDs / HoD Offices may manage. Visible to all roles.
- **Regulations.** Each program may have one regulation document per academic year (`ProgramRegulation`, PDF ≤ 5 MB). The active regulation is the one whose effective-AY is the most recent on or before the current AY. Older regulations remain visible for historical reference.
- **Scheme of Instruction.** For each AY, a scheme records the total number of semesters and, for each semester, the courses on offer (`ProgramSchemeOfInstruction` + `ProgramSchemeCourse`). Electives may be grouped into baskets. The scheme drives course-allocation lookups in the Department module.
- **Specialisations.** `ProgramSpecialisation` carries the code, name and description of any specialisation a program offers; visible to all.
- **Exit Levels.** For programs with multiple exit levels (NEP-style), each exit point records the semester after which exit is permitted and the recognition awarded (certificate / diploma / degree / hons / hons with research).

9.12 Campus Module

Sub-modules: People Directory, Infrastructure, CIE Scheduling, Campus Events, Sports & Culturals, Clubs, Maintenance Complaints, Meetings, Awards & Recognitions, Waste Disposal, Green & Sustainability Initiatives, Systems & Procedures, Approval-driven budget/finance requests.

Owners: Director / Deputy Director / Director Office (manage); all roles view where permitted.

Selected behaviours:

- **People directory.** Each campus exposes a directory of all students, faculty and non-teaching staff currently associated with it. The directory shows display fields only (name, roll/employee ID, designation/program, department, contact email, photo); sensitive PII (Aadhaar, PAN, full address, encrypted documents) is excluded. Visible to all authenticated roles. Searchable and filterable; exportable by Director / Registrar Office.
- *Infrastructure* items have generic name, code, capacity, optional impact, exactly two geo-tagged photos (≤ 2 MB each), one optional impact-proof PDF (≤ 5 MB). A pre-seeded list of generic items is configurable; additional items can be added.
- Items that are simultaneously *green-and-sustainability* relevant maintain bi-directional reflection between the Infrastructure and Green Initiatives sub-modules.

- *Maintenance complaints* include the 10-day reminder loop: if not closed, Director, Deputy Director and Director Office receive an email reminder; reminder count auto-increments on the record.
- *Events / Sports & Cultural / Awards* all support photo + PDF uploads with stated size caps, AY-scoped, downloadable individually or merged per AY into Director-letterhead DOCX.
- *Clubs*: Constitution model with mentor / president / VP / secretary / members; tenure-end alerts at T-30 and T-5 days; activities model captures every required field including outcome, attendance proof, photos. Activity report generated as DOCX with placeholders for mentor and president signatures.
- **Mental-well-being club counsellor validation.** For activities under a club of type *mental well-being*, if the activity is a *counselling* session, the resource person must be one of the campus's currently-configured mental-health counsellors (per the Configuration module). The submission form denies save with a clear error if the configured roster does not contain the chosen resource person, or if the configured roster has no entry valid for the current AY. The error message asks the user to update the Configuration module first.
- *Meetings*: schedule → auto-announcement + email to invitees → HoD/Director uploads minutes file (DOCX or PDF) using the MoM template; attached recordings (if any) are uploaded as file assets, not produced by the system.
- *CIE for first/second UG year*: configured here, auto-projected into department CIE schedules.
- **Waste Disposal sub-module.** Managed by Director / Deputy Director / Director Office; viewable by all. Each disposal record has the following fields:
 - *Title* of the disposal episode.
 - *Type of waste*: e-waste, bio waste, bio-medical waste, hazardous chemical waste, radioactive waste, liquid waste, paper waste, solid waste, other.
 - *Stored since* (date the waste began accumulating).
 - *Disposed on* (date of disposal).
 - *Quantity* (free-text with unit; e.g. “250 kg” or “12 cartons”).
 - *Department helped* (optional; if the campus disposed waste on behalf of a department, the entry is auto-reflected in that department's waste-disposal sub-module).
 - *Authorised vendor / agency name*.
 - *Vendor authorisation & disposal proof*: a single PDF upload (≤ 2 MB) clearly showing the vendor is authorised and the waste was disposed.
- Approval-driven requests (annual budget, event finance, emergency finance) flow through the Approval module; the budget-heading prompt described in the Approval module is presented at submission.

9.13 CESRC Module

Purpose: The Community Engagement & Social Responsibility Cell coordinates the University's outreach, community-service and social-responsibility activities. The original requirements list this as a non-deferred module (it is not starred), so v3 brings it into active scope at a lightweight specification, parallel to the Campus module.

Owners: CESRC Chairperson, CESRC Coordinator, CESRC Office (manage); all roles view where permitted; CESRC Coordinator may compose announcements per the Announcement module.

Sub-modules: Activities, Partner Organisations, Reports.

Behaviours:

- **Activities.** Each activity has a title, type (community outreach, awareness drive, social initiative, environmental drive, other), date, lead faculty, partner organisation (optional,

drawn from Partner Organisations), participants (filterable students / faculty / staff drop-downs from the people modules), location, brief description, key outcomes, max 2 geotagged photos (≤ 2 MB each), one attendance-proof PDF (≤ 2 MB). AY-scoped; immutable on rollover.

- **Partner organisations.** A simple register of external bodies (NGOs, government bodies, community groups) with whom CESRC engages: name, type, contact, MoU reference (optional, links to the Department or Campus MoU record if one exists).
- **Reports.** Individual activity reports and merged AY-level reports are downloadable as DOCX merged with the CESRC Coordinator's letterhead (configured in the Configuration module).
- Approval-driven requests (event finance, emergency finance) flow through the Approval module with the budget-heading prompt.

9.14 Attendance Module

Three interfaces, one source of truth.

- **Warden / Warden Office:** day-level student leave / permission rows.
- **Faculty:** per-course, per-period attendance sheets, projected from the saved department timetable; warden-marked leaves pre-fill.
- **Director / Deputy / Office:** per-student per-course rollup (days held / days absent / %); $< 85\%$ rendered red.
- **Student:** read-only personal view; per-course % live throughout the semester.

Edit precedence rule: faculty entry wins over warden entry on a date that has passed; warden may pre-fill or correct only where faculty has not yet edited.

9.15 Examination Module

CoE / Deputy CoE / CoE Office enter results per student per semester: SGPA, CGPA (where applicable), progression flag, recognition (certificate / diploma / degree / Hons / Hons-with-research) on *passed out*. The progression flag drives the Student-module class transition.

9.16 Reports Module

A reporting layer over the operational schema. Pre-built report templates per stakeholder + an ad-hoc query builder restricted to allowed columns per role. Outputs as CSV / Excel / PDF / DOCX; long reports run async and notify via in-app + email.

9.17 Modules Deferred (per source requirements)

Not in current scope. Listed for completeness so their contracts are not violated by current work: Admissions, Alumni, Campus Hostel, Centres for Excellence, CRIF, Finance, IQAC, LMS, Library, Non-teaching Staff, Placement, Statutory Committees, Tools.

10 Module Dashboards

In plain language: When a user logs in, the home page they see is their dashboard. Different roles see different dashboards. This section sets the contract for what each role's dashboard shows so that the home page is genuinely useful, not a generic landing page.

Every dashboard is composed from a small library of standard *widgets*; each module contributes a fixed set of widgets that the dashboard renderer arranges into the user's home page based on their role(s). A widget's visibility is governed by the same permission system as the rest of the application; if a widget's underlying permission is denied, the widget is silently omitted.

10.1 Standard Widgets

- **Recent announcements.** Top three announcements visible to the user, sorted per the priority engine (§9).
- **Pending approvals (as approver).** Approval requests waiting on the user, oldest first, with a single-click link to act.
- **My in-flight requests.** Approval requests the user has raised that are still open; shows the current stage and the approver currently sitting on it.
- **Calendar at a glance.** Today + the next seven days from the academic calendar, scoped to entries the user can see.
- **Notifications inbox.** Last five unread in-app notifications.

10.2 Per-Role Dashboards

Role	Dashboard widgets (in addition to the standard widgets)
Student	My attendance % per course (red if < 85%); upcoming CIEs; latest results; my mentor; recent feedback for me; my registered VAC and SWAYAM courses.
Faculty	My courses this semester; my workload summary; my pending leave; latest publications I've added; my mentees (if class teacher / mentor); my latest publication-DOI lookups.
HoD / AHoD / HoD Office	Department course allocation status (% of courses allocated); pending VAC requests in my channel; faculty in my department with pending workload updates; upcoming BoS meeting; departmental events this month.
Class Teacher	My class roster; class attendance summary; upcoming CIEs for my class; pending parental feedback drafts.
Director / Deputy Director / Director Office	Maintenance complaints open in my campus, sorted by age (red if > 10 days); upcoming campus events; campus people directory shortcut; pending campus-finance requests; CIE schedule for UG-1/2.
Dean (Sciences / HSS / LL / MC)	Departments under my school: status of vision & mission, regulations, schemes of instruction.
Dean of Academic Affairs	Programs by status; pending VAC final approvals; non-owned-course allocations needing review.

Role	Dashboard widgets (in addition to the standard widgets)
Dean of Student Welfare	Mental-health counsellor coverage by campus; clubs whose tenures expire in the next 30 days; pending welfare-related approvals.
Registrar / Deputy Registrar / Registrar Office	Academic-year status (open/locked); calendar fill status; bulk-import errors awaiting attention; faculty profiles incomplete after first login; student category counts AY-by-AY.
CoE / Deputy CoE / CoE Office	Examination schedule; result-entry progress per program/semester; progression decisions pending.
IQAC Coordinator / Deputy / Office	Calendar fill status; BoS / MoM / VAC template currency; document coverage for every department's vision & mission; per-department workload export shortcut.
CESRC Chairperson / Coordinator / Office	CESRC activities AY-by-AY; partner organisation register; pending CESRC finance requests.
Placement Officer	Placement-related approval requests (when Placement module activates; widget reserved).
HR Head / HR Office	Pending leave approvals at HR level; pending FDP / WFH approvals; staff onboarding pipeline.
VC / VC Office	University-wide pending approvals; campus-by-campus complaint summary; calendar fill status; recent very-important announcements; audit log highlights (auth events, role changes).
System Admin	System health: failed jobs, security scan summary, backup status, recent hard-deletes (always zero or near-zero), audit log search shortcut.

10.3 Implementation Notes

- Widgets render with skeleton state within 250 ms; data hydrates progressively.
- Widgets have a hard ceiling of one server round-trip each; expensive aggregates are pre-computed by Celery jobs and read from cache tables.
- A user with multiple roles sees the union of widgets for those roles, deduplicated.
- Per-role dashboards are templated; no role's dashboard is hand-coded as a one-off page.

11 Leave Rules Module

In plain language: This module encodes the University's official leave rules. Every staff member can apply for leave from this module; the right approvers receive each application in the right order; the system tracks everyone's leave balances; and the rules below cannot be bypassed — not even by a system administrator. This is the most rule-heavy part of the system, so it has its own section.

The Leave Rules Module is a specialised instance of the Approval engine (§9) plus its own balance accounting and an enforced rules engine. The rules below are encoded directly from the *Rules of SSSIHL, 2020*, Section XXVIII.

11.1 Leave Types Supported

1. Casual Leave (CL)
2. Special Casual Leave (SCL)
3. Earned Leave (EL)
4. Half-pay Leave (HPL)
5. Commuted Leave (CML)
6. Extraordinary Leave (EOL)
7. Maternity Leave (ML)
8. Study Leave (SL)

11.2 Universal Rules (applied to every leave type)

- Leave cannot be claimed as a matter of right; the sanctioning authority can refuse, revoke or modify the duration, particularly on examination, BoS, or other academic exigencies.
- The sanctioning authority shall grant only the type of leave applied for.
- Sundays and declared holidays may be prefixed/suffixed and are excluded from the leave count, with the balance treated as on leave account.
- An employee on leave shall not take up any other employment without permission; doing so causes leave salary to cease and triggers disciplinary action.
- Resumption of duty before expiry of leave requires the sanctioning authority's permission.
- Absence after expiry of leave is unauthorised; intervening Sundays/holidays are counted in the over-stay; the over-stay is treated as Extraordinary Leave.
- Leave shall not be granted to an employee likely to be dismissed.
- No leave shall be granted beyond the date on which the employee must retire.
- Any leave *except Casual* may be combined with any other kind of leave.
- Per-month leave statement is to be made available by the 15th of the next month; the system will publish this to HR and the employee.
- No member of staff should leave Headquarters without prior permission of the leave sanctioning authority.

11.3 Casual Leave (CL) — detailed rules

- Annual entitlement: 10 days for vacation employees, 12 days for non-vacation employees.
- Minimum CL granted: half a day.
- Maximum CL at one stretch: 7 days; including prefixed/suffixed Sundays and holidays the stretch must not exceed 10 days.
- Late attendance for 3 days in a month $\Rightarrow$ forfeiture of 1 day's CL (auto-applied).
- Sundays and holidays falling *within* a CL period are not counted as leave.
- Availing CL without prior permission is treated as unauthorised absence unless regularised by the competent authority for special reasons.
- Un-availed CL on 31 December lapses each year (auto-applied).
- In the year of joining, entitlement is proportionate to the months of service.
- In the year of leaving, excess CL availed (above proportion) is recovered against HPL/CML/EL or, if none due, recovered from final salary payment.

11.4 Special Casual Leave (SCL) — detailed rules

- Annual limit: 8 days for teaching staff, 12 days for non-teaching staff.
- Granted for presenting a paper in symposia, conferences, seminars, or other purposes the Institute deems necessary.
- Treated as leave on duty.

11.5 Earned Leave (EL) — detailed rules

- For non-vacation employees (regular and temporary): 2.5 days credited per completed month of service. Credited on 1 January and 1 July every year. Fractions rounded to the nearest day.
- Maximum accumulation: 300 days.
- Maximum availed at one time: 60 days; exceptions allowed for higher study, training, leave with medical certificate, or when the leave (or any portion) is spent outside India.
- For partial leave outside India, EL beyond 120 days is conditional on the in-India portion not exceeding 120 days in aggregate.
- For teachers (vacation employees), EL admissible is the sum of:
 - (a) 1/30th of actual service including vacation; *plus*
 - (b) 1/3rd of any vacation period during which the teacher was required to perform duty (excluding admission-test and examination duty); *plus*
 - (c) 1/3rd of 2 weeks (the vacation-curtailment credit).
- If a teacher remains on duty for the entire vacation, entitlement is 30 days EL; for partial vacation duty, the entitlement is proportionate.
- Vacation, if combined with EL, counts toward the maximum-on-average-pay calculation for the leave period.
- For computing actual service eligible for EL, all leave except Casual, Special Casual and duty leave is excluded.
- Earned leave preparatory to retirement, if refused in the Institute's interest, is treated as service extension.
- Encashment of EL at retirement: teaching (vacation) staff cannot encash. Non-vacation teaching and administrative staff can encash with the cap that EL plus HPL together $\leq$ 300 days. Salary for encashment excludes HRA and CCA.

11.6 Half-pay Leave (HPL) and Commuted Leave (CML) — detailed rules

- HPL admissible at 5/3 days per completed month of service.
- Maximum HPL accumulation: 180 days. Credited on 1 January and 1 July.
- HPL may be granted with or without medical certificate.
- CML may be granted with medical certificate, subject to:
 - (a) Maximum CML accumulation: 90 days.
 - (b) Maximum CML at one time: 60 days.
 - (c) Combination of EL and CML permissible up to 120 days.
 - (d) Whenever CML is granted, twice the days availed are debited against HPL due.
 - (e) Maximum CML permitted with medical certificate is 90 days during entire service.
 - (f) Application requires a medical certificate from a Registered Medical Practitioner not below the rank of Civil Surgeon; return-to-duty requires a fitness certificate from the same.
- Salary: full salary on EL or CML; half salary on HPL.

11.7 Extraordinary Leave (EOL) — detailed rules

- Without salary.
- Granted when (i) no other leave is admissible, or (ii) the employee, while otherwise eligible, requests it in writing.
- Maximum 180 days at one time; aggregate over entire service shall not exceed 1 year.
- Authority may retrospectively commute absence-without-leave into EOL for valid reasons.

11.8 Maternity Leave (ML) — detailed rules

- Available to a married woman employee with at least 1 year of service.
- Standard entitlement: 26 weeks, of which not more than 8 weeks shall precede the expected delivery date.
- For a woman with 2 or more surviving children: 12 weeks, of which not more than 6 weeks precede the expected delivery date.
- May be combined with other leave (except Casual) supported by a medical certificate.
- May be granted in the case of miscarriage.

11.9 Study Leave (SL) — detailed rules

- Maximum duration: 1 year.
- Generally admissible to teaching staff in the Post-graduate Department.
- Ordinarily admissible only to employees with 5 years of service.
- Granted only if the Institute is satisfied (a) the object benefits both the employee and the Institute, and (b) the absence can be covered without detriment to the Institute.
- Requires a stamped-paper bond agreeing to serve the Institute for at least 5 years after return; default leads to refund of salary drawn during study leave (or such portion as the Institute determines), with interest at 11% per annum, within one month of default; medical unfitness or death exempt.
- Not ordinarily admissible to employees over 45 years of age.
- Counts as service for promotion but not for earning any leave.
- Salary as per last drawn; any stipend / scholarship / remuneration earned during study leave is deducted.
- Cannot be claimed as a matter of right; the Institute may refuse it.
- Requires submission of a complete report on work done, on return to duty.
- No appointment elsewhere within the bond's 5-year period without permission.

11.10 Sanctioning Authority Matrix

Leave Type	Applicant Category	Sanctioning Authority
CL	Registrar, CoE	Vice-Chancellor
CL	Professors, Directors, Associate Professors (Readers)	Vice-Chancellor
CL	Lecturers, Lecturer-Senior, Lecturer-Selection Grade, Departmental staff	Director of the Campus
CL	Staff working in VC's office / Finance Officer's office / Registrar's office / CoE's office	VC / FO / Registrar / CoE respectively
CL	Staff working in Director's office in a campus	Director of the Campus
SCL	Any employee	Vice-Chancellor (or any authority delegated by him); routed through the Director (recommending only)
EL, HPL, CML, ML	Staff in VC / FO / Registrar / CoE office	VC / FO / Registrar / CoE respectively

Leave Type	Applicant Category	Sanctioning Authority
EL, HPL, CML, ML	Lecturers, Lecturer-Senior, Lecturer-Selection Grade, Departmental staff in a Campus	Director of the Campus
EL, HPL, CML, ML	Professors, Directors, Readers (in any Campus)	Vice-Chancellor (or specifically authorised authority)
EOL, SL	All employees	Vice-Chancellor (may delegate)
Director's own leave (any type)	Director	Vice-Chancellor; Director must invariably recommend the faculty member who will be in charge during absence

11.11 Special Cases

- **Honorary non-teaching staff:** 30 days leave/year + CL on the same terms as regular employees.
- **Honorary teaching staff:** 10 days leave/year + CL on the same terms as regular employees.
- **Visiting fellows:** CL on a proportionate basis based on length of service. EOL needs prior VC approval; in real emergencies, post-facto ratification.
- **Re-employed superannuated teaching staff:** 10 days CL/year, 10 days medical leave/year (with certificate), and 20 days additional leave with full honorarium. Not carried forward across terms.
- **Re-employed superannuated non-teaching staff:** 8 days CL/year, 10 days full-pay leave with medical certificate, 30 days paid leave/year without medical certificate. Not carried forward across terms.

11.12 Data Model for Leave

```
class LeaveBalance(TimestampedSoftDelete, table=True):
    """Per-employee, per-leave-type running balance, snapshot per AY."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    employee_user_id: UUID = Field(foreign_key='user.id', index=True)
    leave_type: str = 'CL' | 'SCL' | 'EL' | 'HPL' | 'CML' | 'EOL' | 'ML' | 'SL'
    academic_year_id: UUID = Field(foreign_key='academicyear.id')
    opening_balance_days: float
    credited_days: float = # +ve credits during the year
    availed_days: float
    forfeited_days: float = # e.g. CL forfeitures from late attendance
    encashed_days: float = 0
    closing_balance_days: float

class LeaveRequest(TimestampedSoftDelete, table=True):
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    requestor_user_id: UUID = Field(foreign_key='user.id', index=True)
    leave_type: str
    starts_on: date
    ends_on: date
    half_day: bool = False = # CL only; first or last day half-day flag
    reason: str
    medical_cert_file_id: UUID | None = Field(foreign_key='fileasset.id')
    fitness_cert_file_id: UUID | None = Field(foreign_key='fileasset.id')
    bond_file_id: UUID | None = Field(foreign_key='fileasset.id') = # Study leave
```

```

address_during_leave: str | None = None
headquarters_left: bool = False
alternate_arrangement: str | None = None
intended_outside_india: bool = False
approval_request_id: UUID = Field(foreign_key='approvalrequest.id', unique=True)
state: str # mirrors approval_request.state
sanctioned_days: float | None = None # may differ from requested if modified
cancellation_reason: str | None = None

class LateAttendanceMarker(TimestampedSoftDelete, table=True):
    """One row per late-attendance occurrence; the rule
    (3 in a month -> forfeit 1 CL day) runs as a Celery job."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    employee_user_id: UUID = Field(foreign_key='user.id', index=True)
    occurred_on: date

class LeaveSanctionAuthorityRule(TimestampedSoftDelete, table=True):
    """Encodes the matrix in section 'Sanctioning Authority Matrix'.
    Loaded from a YAML seed file; live-editable by System Admin
    under tight audit. Rule resolution returns the required
    channel of approver role codes for a (leave_type, applicant) pair."""
    id: UUID = Field(primary_key=True, default_factory=uuid4)
    leave_type: str
    applicant_role_code: str
    applicant_designation_pattern: str | None = None
    applicant_office_role_code: str | None = None
    sanctioner_role_code: str
    is_recommend_only: bool = False # for SCL routing through Director
    notes: str | None = None

```

Listing 7: Leave-specific tables.

11.13 Behavioural Rules (engine)

A pure-Python rules engine (`services/leave_rules.py`) implements these as deterministic functions, called by both the request submission and the request approval steps:

- `compute_leave_days(starts_on, ends_on, leave_type, half_day, ay_id)` — returns the chargeable day count. Sunday handling: standard. Declared-holiday handling: queries the `Holiday` table for rows matching the given academic year and applies the per-type rule (Sundays & holidays falling *within* CL are not counted as leave; for other leave types, prefixed/suffixed Sundays & holidays at either end are excluded from the leave count). Single source of truth for “what is a working day” is the `Holiday` table for the active AY.
- `check_balance(user, leave_type, days)` — denies submission if the closing balance after this request would go negative (except for EOL, which has no balance).
- `check_max_at_a_time(leave_type, days, payload)` — enforces the per-type ceilings; allows EL beyond 60 only with the documented exception flag.
- `check_combination(leave_type, overlapping_requests)` — denies CL combined with any other leave type.
- `check_eligibility(user, leave_type)` — maternity (women, $\geq 1y$ service); study (5y service, age ordinarily ≤ 45 , PG dept).
- `resolve_channel(user, leave_type)` — returns the approver-role channel from `LeaveSanctionAuthorityRule` for Director’s own leave, automatically requires the in-charge-faculty designation field on the request.

- `forfeit_late_cl()` — nightly Celery job; for any employee with ≥ 3 `LateAttendanceMarker` rows in the previous month, debits 1 CL day with a `forfeited_days` entry and audits.
- `lapse_unavailed_cl()` — 31-December Celery job; for every employee, sets unavailed CL closing balance to zero and rolls into the next AY’s opening balance (i.e. does not carry forward).
- `credit_periodic()` — 1-January and 1-July Celery jobs; credits EL and HPL according to the per-type rules, including the teacher-specific formula.
- `handle_overstay(request, return_on)` — on detection of return-to-duty after sanctioned end date, auto-creates an EOL request retroactively for the over-stay including intervening Sundays/holidays, audits, and notifies the sanctioning authority.

11.14 User Experience

- Each employee sees a single “My Leave” page: current balances per type, history, in-flight requests, and “Apply for Leave.”
- “Apply for Leave” is a single guided form: pick type → pick dates → system shows the chargeable days → shows the resolved channel of approvers → requestor confirms.
- For medical-grounds CML, the medical-certificate upload is required at submission; for return-to-duty, the fitness certificate is required.
- For Study Leave, the bond document upload is required at submission.
- For travel outside Headquarters, the address-during-leave and alternate-arrangement fields are required.
- Cancellation by requestor is allowed at any time before any approver has acted; withdrawal is allowed at any later stage; both are audited.
- Sanctioning authority can sanction *partial* duration (modify) with a comment; the sanctioned span replaces the requested span and the balance debit reflects the sanctioned span.

11.15 Seed YAML for the Sanctioning Authority Matrix

The matrix is loaded at first boot from a single source-of-truth YAML file. System Admin can edit live, under audit. Below is an extract showing the structure; the full file is delivered with the seed data and covers every row in the matrix table above.

```
# Each entry resolves a (leave_type, applicant) pair to a sanctioning role.
# applicant fields:
#   role_code           - exact role match (e.g. 'PROFESSOR')
#   designation_pattern - optional regex on Faculty.designation
#   office_role_code    - optional; matches staff working in that role's
#   office
#   is_vacation_employee - optional; True | False | null

- leave_type: CL
  applicant: { role_code: PROFESSOR }
  sanctioner: { role_code: VC }
  notes: Per Rules of SSSIHL 2020 Sec. XXVIII clause 22.I.ii

- leave_type: CL
  applicant: { role_code: ASSOC_PROFESSOR }
  sanctioner: { role_code: VC }

- leave_type: CL
  applicant:
    role_code: FACULTY
    designation_pattern: '^(Lecturer|Lecturer-Senior|Lecturer-Selection Grade)$'
  sanctioner: { role_code: DIRECTOR, scope: campus }
```

```

notes: Sec. XXVIII clause 22.I.iii

- leave_type: CL
  applicant: { role_code: STAFF, office_role_code: VC }
  sanctioner: { role_code: VC }

- leave_type: SCL
  applicant: { role_code: '*' } # any
  sanctioner: { role_code: VC }
  recommend_only_via: { role_code: DIRECTOR, scope: campus }
  notes: Sec. XXVIII clause 22.II

- leave_type: EL
  applicant: { role_code: PROFESSOR }
  sanctioner: { role_code: VC }
  notes: Sec. XXVIII clause 23.b.ii

- leave_type: [EOL, SL]
  applicant: { role_code: '*' }
  sanctioner: { role_code: VC, may_delegate: true }
  notes: Sec. XXVIII clause 24

- leave_type: '*' # any leave type, when applicant is the
  Director
  applicant: { role_code: DIRECTOR, scope: campus }
  sanctioner: { role_code: VC }
  requires_in_charge_designation: true
  notes: Sec. XXVIII clause 23.b.ii Note

```

Listing 8: Extract from `seeds/leave_sanction_matrix.yaml`.

11.16 Acceptance Criteria for the Leave Module

- Synthetic-employee suite: 20 representative employees (mix of vacation/non-vacation, teaching/admin, men/women, joining-year/established) demonstrate every rule in this section, including credit jobs, forfeiture, lapse, encashment caps, the maternity-with-2-children cap, and study-leave-bond enforcement.
- Sanctioning matrix coverage: a property-based test enumerates every (`leave_type`, `applicant_role`) pair in the seed and asserts the channel resolved matches the matrix.
- Audit footprint: every state change on every `LeaveRequest` is in `AuditLog` with a diff payload.
- No path bypasses `check_balance` or `check_eligibility` — enforced by tests that try direct service-layer calls with crafted payloads.

12 Implementation Order & Milestone Gates

In plain language: Below is the order in which the work will be done. Each step ends in a “gate” — a checklist that must be fully ticked before the next step begins. This is the discipline that makes the project safe to deliver. Do not let scope move forward unless its gate is passed.

The order below is binding. A gate is a yes/no decision, not a discussion. The administrator is expected to attend the demonstration at each gate and tick the checklist with the engineering

lead.

#	Milestone	Contents
M0	Foundations	Repo bootstrap; Reflex skeleton; PostgreSQL schema; Alembic migrations; structured logging; CI; pre-commit; theme provider with the Puttaparthi Saffron palette; Docker build; docker-compose for dev including Mailpit as the SMTP sandbox; scripts/seed.py producing synthetic mock data. <i>Gate:</i> clean pipeline; app boots; theme renders consistently; mock seed runs end-to-end.
M1	Authentication	Login, logout, change password, forgot password, sessions, 7-day idle expiry, audit of auth events, rate limiting, lockout. All outbound mail captured by Mailpit. <i>Gate:</i> Playwright auth suite green; brute-force test blocked; password-reset email visible in Mailpit.
M2	Admin (users + roles + permissions)	User CRUD with role assignment; role CRUD; permission matrix UI; bulk import; password auto-gen + email; basic-user fallback. <i>Gate:</i> System Admin can construct any role and verify scoped permissions.
M3	Configuration — Org core	Campus / School / Department (with school_id) / SubDepartment / Centre / Program (basic + ProgramOutcome + ProgramRegulation + ProgramSchemeOfInstruction + ProgramSpecialisation + ProgramExitLevel) / Course (basic) seeding + management; vision and mission management for university and departments; class timings; working days config. <i>Gate:</i> all four campuses, four schools, ten departments, sub-departments, centres seeded; one program seeded with full PEO/PO/PSO/regulation/scheme/exit-level data.
M4	Configuration — AY & Calendar	AcademicYear; Holiday; Calendar collaboration chain (Registrar → IQAC → others); rollover lock job; calendar exports; sports & cultural calendar by Directors. <i>Gate:</i> two AYs configured, one locked, edits denied.
M5	Configuration — Identity attachments	Role-based emails; letterheads; templates (BoS, MoM, VAC); mental-health counsellors; faculty mentor assignments; class-teacher assignments; non-owned-courses; UG timetable; approval-process configurations (incl. CPC fund-release); announcement-composer roster; StudentCategoryCount per AY. <i>Gate:</i> every artefact required by downstream modules exists in Configuration.
M6	Audit Module UI	Filter / search / export of AuditLog . <i>Gate:</i> System Admin can locate any historical action by 2 filters in < 5 seconds.

#	Milestone	Contents
M7	Approval Request Engine	Generic approval runtime; cancellation/withdrawal/extension; attachment caps; notifications; finance budget-heading prompt for is.finance=true processes; CPC fund-release process fully wired. <i>Gate:</i> VAC sample flow operates end-to-end; CPC fund-release flow operates end-to-end.
M8	Leave Rules Module	Full encoding of §11: balance accounting; rules engine (compute_leave_days consumes Holiday); sanctioning matrix loaded from YAML; credit/lapse/forfeit Celery jobs; My-Leave UI. <i>Gate:</i> 20-employee synthetic suite passes; every rule in §11 demonstrated.
M9	Announcement Module	Composer roster from M5; priority engine; targeted delivery; auto-announcement hooks. <i>Gate:</i> priority and 2-day window correct under unit tests with mocked clock.
M10	Faculty Module (core + workload + requests)	Profile, education, experience, expertise, documents, parametric request types. <i>Gate:</i> a faculty user can fully populate their record and raise an FDP request through the M7 engine; leave requests use the M8 module.
M11	Faculty Module (research)	Publications, patents, IPRs, awards, collaborations, projects, consultancy, e-content, PhD guidance, research guidance — with DOI/CrossRef integration and co-author cross-tagging. <i>Gate:</i> DOI-driven create works on a curated sample of 20 real DOIs; cross-tag round-trips correctly.
M12	Student Module & Research Scholars Module	Profiles, SWAYAM log, parental feedback channel (full 5-bullet behaviour); scholar mirror of research-related faculty submodels. <i>Gate:</i> parental feedback delivered by Mailpit + archived; correction-feedback flow demonstrated.
M13	Course & Program & Department Modules — allocation & timetable	Full course detail; full Program management UI (PEOs/POs/PSOs, regulations, schemes, specialisations, exit levels); per-semester allocation; timetable with conflict detection; workload propagation; merged exports. <i>Gate:</i> allocation → timetable → workload chain produces consistent state; program scheme drives course-allocation lookups.
M14	Department Module — BoS, events, VAC, MoUs, CIE, requests	All remaining department features. Minutes uploaded as files; no AI generation. <i>Gate:</i> BoS workflow demo; VAC end-to-end demo.
M15	Campus Module	People directory; infrastructure with bidirectional Green-Initiatives reflection; waste disposal (full enumerated fields); systems & procedures; maintenance complaints with 10-day reminders; events; sports/culturals; clubs (with tenure alerts and mental-well-being counsellor validation); meetings (file-attached minutes); awards; CIE scheduling for UG-1/2; approval-driven finance requests. <i>Gate:</i> every sub-module green per its own checklist; counsellor-validation rule blocks invalid entries.

#	Milestone	Contents
M16	CESRC Module & Module Dashboards	CESRC activities, partner organisations, AY-scoped reports; per-role dashboards composed from the standard widget library. <i>Gate:</i> CESRC AY-cycle demo; each role’s dashboard shows its required widgets and respects permissions.
M17	Attendance Module	Three interfaces; precedence rule; live student view; auto-driven by saved timetable. <i>Gate:</i> simulation test with synthetic week passes.
M18	Examination Module	Result entry; progression engine drives Student class transitions and active-flag changes. <i>Gate:</i> synthetic semester rollover correct for each progression value.
M19	Reports Module	Pre-built reports + ad-hoc builder + async generation. <i>Gate:</i> all pre-buils exportable in all four formats.
M20	Hardening	Security review (OWASP ASVS L2 checklist), load test, accessibility audit, theme polish, backup/restore drill, runbook, production SMTP cutover with DKIM/SPF/DMARC . <i>Gate:</i> sign-off by sponsor for production.

On the order

The Leave Rules Module (M8) is placed immediately after the generic Approval Engine (M7) and before any module that needs to raise leave requests (Faculty M10 onwards). This ordering matters: it gives the engine time to be tested in isolation, and it means every later module that surfaces a “leave” link is wiring into a fully-tested engine.

13 Testing Strategy

In plain language: Tests are how we prove the system actually works — on the day it’s built, and on every day after. The system will have four kinds of automatic tests, all of which run on every change. No milestone is accepted unless all its tests pass.

13.1 Levels of Test

1. **Unit tests (pytest).** For every service function, repository, rules-engine function, validator, and utility. These are fast (milliseconds) and run on every commit. They prove individual pieces are correct in isolation.
2. **Integration tests (pytest + a real PostgreSQL container).** For every module’s service-and-repository combinations; verify that the database operations behave as intended (constraints fire, cascades work, soft-delete hides rows, audit rows appear, transactions roll back on failure).
3. **End-to-end tests (Playwright for Python).** For every major user journey: log in, change password, submit a leave request, approve a leave request, file a maintenance complaint, configure the academic year, etc. These run a real browser against a fully-wired application.

4. **Property-based tests (hypothesis).** For all rule-heavy code: leave-day computation, balance arithmetic, the sanctioning matrix resolution, the announcement priority order. Hypothesis generates many random valid inputs and asserts properties hold across all of them.

13.2 Specific Test Requirements per Concern

- **Permissions:** for every protected endpoint, two tests — one principal who is permitted (succeeds), one who is not (denied with the correct error). A property-based test enumerates a sample of (`role`, `scope`, `target`) combinations and asserts `can()` returns the expected verdict.
- **Audit:** for every state-changing operation, a test that performs the operation and asserts a corresponding row appears in `AuditLog` with the expected actor, action, resource, and a non-null `diff_json`.
- **Soft and hard delete:** for every model that supports them, tests that prove (a) soft-delete hides from default queries but the row remains; (b) only System Admin can hard-delete; (c) hard-delete is allowed only on rows that are already soft-deleted; (d) confirmation gate is enforced at the API layer too, not only in the UI.
- **AY immutability:** for every AY-scoped model, a test that locks a year and asserts edits and deletes are denied with the right error.
- **Calendar collaboration:** a test that runs the Registrar → IQAC → others chain on a synthetic AY and asserts each role can edit only their own entries.
- **Leave rules:** the entire rules-engine surface is property-tested; the synthetic-20 suite from M8 is the integration baseline.
- **Announcements:** a test with a mocked clock that advances day-by-day and asserts the priority/2-day-window rule resolves correctly across boundary conditions (Sundays, declared holidays, very-important overrides).
- **File uploads:** tests for size limits, MIME validation, ClamAV scan invocation (mocked), checksum recording, deduplication.
- **DOI lookup:** CrossRef and DataCite calls mocked at the HTTP layer; tests for happy path, no-DOI, DOI-not-found, network timeout, malformed response.
- **Migrations:** every Alembic migration is run forward then backward against a clone of the previous schema in CI; an additional test seeds the previous schema, applies the migration, asserts data integrity.
- **Performance smoke:** a synthetic dataset of 5 000 rows is loaded, every list view is rendered, and the render time is asserted under 1.5 s (with allowance for CI hardware variance).
- **Accessibility:** `axe-core` run via Playwright on every page; zero critical violations is the gate.
- **Security:** `bandit`, `pip-audit`, `trivy` run in CI; high/critical findings block the merge.

13.3 Coverage Targets

- Service layer: $\geq 85\%$ line coverage and $\geq 80\%$ branch coverage.
- Rules engine (leave, approvals, attendance, examination): $\geq 95\%$ line, $\geq 90\%$ branch.
- Repository layer: $\geq 80\%$.
- UI states: $\geq 70\%$.

- E2E: every critical user journey has at least one Playwright scenario.

13.4 Continuous Integration

- Every push triggers: lint → typecheck → unit → integration → E2E → security scans → image build.
- Pull requests are merge-blocked unless the entire pipeline is green and at least one human reviewer (the engineering lead) has approved.
- Nightly: full E2E suite, accessibility scans, performance smoke at 5 000-row scale.
- Weekly: dependency-update PR via Dependabot/Renovate; security scan baseline refresh.

13.5 Manual Test Plan

At each milestone gate, in addition to the automated suite, the following is performed manually with the administrator present:

1. Live demonstration of every behaviour listed in this RFP for that milestone.
2. Spot-check on at least three random screens at the four target widths (360, 768, 1280, 1920 px).
3. Spot-check of the audit log: perform a known action, observe the audit row appear in real time.
4. Spot-check of permissions: log in as a less-privileged role and attempt a forbidden action; observe denial.

14 Acceptance Criteria & Quality Bar

In plain language: For every milestone, this is the checklist that turns “done” into “accepted.” The administrator can use this list as their own checklist during the demonstration.

14.1 Per-Milestone Gate Checklist

1. **Functional:** every behaviour written in this RFP for the milestone is implemented and demonstrated end-to-end.
2. **Schema:** migrations apply cleanly forward and reverse cleanly back on a dev clone of production; seed data is idempotent.
3. **Permissions:** every page and event handler is protected; tested with at least one permitted and one denied principal.
4. **Audit:** every state-changing action produces a row in `AuditLog` with a diff payload.
5. **Soft + hard delete:** both implemented; hard delete reachable only by System Admin; both confirmed by dialog.
6. **UX:** responsive at 360, 768, 1280, 1920 px; theme renders consistently on every page.
7. **Tests:** unit, integration, Playwright E2E added; coverage thresholds met; property-based tests added for rule-heavy code.
8. **Documentation:** docs/modules/<module>.md updated.

9. **Security:** `bandit` / `pip-audit` / `trivy` clean; OWASP-relevant items in checklist marked.
10. **Performance:** list views meet the 1.5s budget at 5 000 rows.

14.2 Programme-Level Bar

- Code coverage thresholds met across the codebase (see §13).
- Accessibility: axe-core scan with 0 critical violations on every page.
- Documentation: a single deployable runbook (`docs/runbook.md`) describes day-2 operations.
- Backups: nightly DB + object-storage backup with monthly tested restore.
- Disaster recovery: $RTO \leq 4$ hours, $RPO \leq 24$ hours (committed at M20).

15 Theme & Visual Identity

In plain language: The system has one elegant theme. All colours, fonts and spacings are defined once, in one place, so that any later change applies everywhere consistently.

15.1 The Single Committed Theme: Puttaparthi Saffron–Indigo–Ivory

Inspired by the saffron-and-indigo character of Sri Sathya Sai Hostel buildings in Puttaparthi, with ivory paper tones for a calm, high-readability surface. The same palette used throughout this document is the palette the application will use.

- Primary structure: indigo #1F3A6E.
- Primary accent: saffron #C75B12.
- Surface: ivory #FAF7F2.
- Body text: slate #2C2C2C.
- Secondary text: muted #6B6B6B.
- Decorative rule: gold #B89B6A.

15.2 Implementation

The theme is a `ThemeContext` of CSS variables consumed by all components. No component hard-codes colours. This means a future decision to add additional themes requires only adding new variable sets and a switcher — no code rework. The administrator may request additional themes after M20.

16 Risks & Best-Available Mitigations

In plain language: Every project has risks. Below, every risk is stated plainly and matched to the best available open-source mitigation. There are no unresolved questions: if v1 left a decision open, v2 has made the conservative choice and explained it.

Risk	Best-available mitigation (committed)
WhatsApp delivery vs. “open-source, no cost” constraint	Resolved by descope. v2 commits to email + in-app notifications only. SMTP is free, reliable, open and has decades of operational tooling. The code remains channel-pluggable, so a future board decision can introduce WhatsApp as a separate engagement if the cost trade-off changes.
Audio recording quality and stability in a Reflex/browser environment	Resolved by descope. v2 commits to allowing meeting recordings as <i>file uploads</i> , made by whichever device the meeting actually used (laptop / phone / dedicated recorder). The system stores them on MinIO, indexed against the meeting record. This is more reliable than browser-side capture and removes a hard engineering risk.
Quality of AI-generated minutes	Resolved by descope. v2 commits to human-drafted minutes uploaded as files. The system provides a rich-text editor in the browser (TipTap or Quill) for those who prefer to compose in-app, then save to the meeting record.
Reflex API churn between versions	Pin Reflex to a single version per milestone. Bump versions only at M-boundaries with a regression sweep. Keep an upgrade-notes file in the repo so each bump is documented.
Audit-table volume	Monthly partitions on <code>auditlog.occurred_at</code> via PostgreSQL declarative partitioning. Cold partitions can be moved to slower disk via a tablespace move. A retention rule (e.g. 7 years online, archived afterwards) is set in M20 with the sponsor.
Permission complexity	Centralise in <code>permissions.can()</code> as a pure function. Property-based tests over a representative sample of (<code>role</code> , <code>scope</code> , <code>target</code>) combinations. Every page and handler uses the decorator; lint check fails the build if a handler is unprotected.
DOI lookup failures	Cache CrossRef responses in a <code>publication_metadata_cache</code> table keyed by DOI; fall back to DataCite; allow manual entry of all fields except impact factor and quartile (which remain “Not Available” rather than risk false data).
Theme accessibility	<code>axe-core</code> runs in CI on every page; the single theme’s contrast ratios are verified at the design-token layer; no merge if a critical violation is introduced.
Locked AY data confusion	UI badges on every AY-scoped record showing “Current AY” or “Previous AY (read-only).” All AY-scoped pages display a one-line banner stating the AY context.
Bulk-import data quality	Two-stage import: validate (preview a CSV against the schema, show every row’s status), then commit. Errors do not commit the row; the user sees a downloadable report of the failures.

Risk	Best-available mitigation (committed)
Email deliverability through development & production	From M1 through M19 , all outbound email is captured by Mailpit (open-source SMTP sandbox) running in the dev/test environment. Mailpit catches every email the application would send so flows like password reset, user creation and approval notifications can be exercised end-to-end without sending real mail. At M20 , production SMTP cuts over to the University's mail relay; the administrator works with the IT team to publish DKIM keys, configure SPF for the relay, and a strict DMARC policy. The application supports DKIM signing via the SMTP layer.
Seed data timing	Real seed data is not on the critical path. A <code>scripts/seed.py</code> generates synthetic-but-realistic mock data covering users, faculty, students, programs, courses and a sample academic year. Development and testing run against this mock from M0 onwards. When the sponsor delivers real data, it loads through the same M5 bulk-import flow that handles real-life additions; the mock is wiped and replaced. There is no scheduling dependency between the build and the sponsor's data preparation.
Backup and disaster recovery	Nightly <code>pg_dump</code> + <code>pgBackRest</code> continuous archive; MinIO bucket replication; quarterly tested restore drill. Documented RTO/RPO at M20.
Onboarding new staff (System Admin successor)	The runbook (<code>docs/runbook.md</code>) is treated as a deliverable. Every operational task (rotate credentials, restore from backup, add a campus, lock an AY) has a step-by-step procedure with screenshots.

Inputs still needed from sponsor, with defaults committed if unanswered

- Statutory committees roster (defer until that module is scheduled).
- ABC-ID validation: defaults to format-only validation in v1; live ABC API integration is a future enhancement.
- Production email-sending domain and DMARC plan: due before M20. Mailpit covers the entire build.
- Real seed data files for users, courses, programs, faculty, students: arrive when ready; mock data covers the build until they do.

None of the above blocks the M0–M8 work; the plan can begin immediately.

17 Feasibility for Claude Code

In plain language: A direct answer to the question: can the AI coding assistant build this? Yes, if the work is broken up exactly as in this document, and if the administrator approves each milestone before the next one starts.

17.1 Bottom Line

Yes, this is achievable with Claude Code — provided the engagement adheres to the milestone gating in §12 and the testing requirements in §13. The descopes in v2 (no WhatsApp, no audio recording, no AI minutes, single theme) eliminate the three highest-friction items from v1 and concentrate the work on Claude Code’s strongest territory.

17.2 What Claude Code Will Do Well

1. Translate the schemas in §8 into SQLAlchemy files, Alembic migrations, repositories and services.
2. Generate consistent CRUD pages across modules from a small set of conventions.
3. Wire up the permission and audit decorators uniformly, once their first instances are reviewed.
4. Implement DOI-driven publication entry (CrossRef + DataCite REST clients, with caching).
5. Build calendar collaboration, approval-engine state machines, attendance precedence rules, examination progression engine.
6. Produce DOCX/PDF/Excel/CSV exports against letterhead templates.
7. Write the test suites that prove all of the above.
8. Encode the leave-rules engine (§11) faithfully with the property-based tests that prove every rule.

17.3 What Needs Human Decision & Review

1. Final visual polish on the single theme — type weights, button radii, table density.
2. Acceptance of every milestone gate.
3. Provision of seed data (the sponsor will supply these on a separate schedule; the system will accept them via the bulk-import mechanism in M5).
4. Final security sign-off at M20.
5. Production rollout: secrets, network, DNS, TLS issuance, backup-restore drill — a human operator’s decision-set.

17.4 How to Drive Claude Code on This Project

1. One milestone, one branch, one PR. Never give Claude Code more than one milestone at a time.
2. Reference this document by section number in every prompt: e.g. “Implement §9 (Maintenance Complaints) per the schema in §8 and the M15 acceptance criteria in §12.”
3. Demand passing tests as part of the deliverable, not after.
4. Run migrations forward and backward in CI on a clone of production schema.
5. Re-read the gate before merging: every gate in §12 is yes/no, not a discussion.

Verdict

With the scoping, gating and human-review discipline above, DURGAM as specified is achievable through a Claude Code engagement. The descopes in v2 and the explicit Leave Rules encoding (the rule-heaviest area of the system) bring the work firmly into Claude Code’s core competency: contract-driven, test-first implementation across a large but well-bounded surface.

18 Appendix A — Seed Data Reference

In plain language: The starting set of campuses, departments, sub-departments and centres. The sponsor will provide the rest of the seed data (users, faculty, students, programs, courses) on a separate schedule; that data will be loaded through the bulk-import mechanism built in milestone M5.

- **Campuses:** Prasanthi Nilayam (PSN), Brindavan (BRN), Muddenahalli/Nandigiri (NDG), Anantapur (ATP).
- **Departments:**
 - Biosciences (DBIO) — ATP, PSN.
 - Chemistry (DCHEM) — ATP, PSN.
 - Education (DEDN) — ATP.
 - Food & Nutritional Sciences (DFNS) — ATP.
 - Humanities & Social Sciences (DHSS) — ATP, BRN, PSN.
 - Languages & Literature (DLL) — all four campuses.
 - Mathematics & Computer Science (DMACS) — all four campuses.
 - Management & Commerce (DMC) — ATP, BRN.
 - Physics (DPHY) — ATP, PSN.
 - Performing Arts (DPA) — PSN.
- **Sub-departments under HSS:** Philosophy (SDPHIL — ATP), Psychology (SDPSY — ATP, PSN), History (SDHIST — PSN, ATP), Political Science (SDPOL — PSN, ATP), Economics (SDECON — BRN, ATP).
- **Sub-departments under L&L:** English (SDENG — all), Telugu (SDTEL — PSN, ATP), Hindi (SDHIN — all), Sanskrit (SDSAN — all).
- **Centres for Excellence:** Mathematical Biology (CMB — PSN), Sri Sathya Sai Studies (CSSS — PSN), Actuarial & Data Sciences (CADS — BRN), Sustainable Development (CSD — PSN).

19 Appendix B — Repository Layout

```
durgam/
  app/
    __init__.py
    durgam.py                # rx.App() entry
    pages/                   # one folder per module
      auth/  admin/  configuration/  audit/  announcements/
      approvals/  leave/  faculty/  students/  research_scholars/
      department/  campus/  attendance/  examination/  reports/
    components/              # reusable UI: tables, forms, dialogs, charts
    states/                  # Reflex State classes per page
    services/                # business logic; never touches the ORM directly
      leave_rules.py
      approvals.py
      attendance.py
      ...
    repositories/            # SQLAlchemy queries
    models/                  # SQLAlchemy domain models
    auth/                    # session, password, permissions
    audit/                   # decorator + writer
```

```

notifications/      # email, in-app channels
storage/            # MinIO/local abstraction
docgen/             # docx, pdf, xlsx generators
integrations/       # crossref, datacite, ...
jobs/               # Celery tasks (credit/lapse/forfeit, reminders)
config.py           # pydantic-settings
main.py
alembic/            # migrations
docs/
  modules/          # one .md per module
  runbook.md
tests/
  unit/  integration/  e2e/  property/
scripts/
  seed.py
  backup.sh
docker/
  Dockerfile.app
  Dockerfile.worker
  docker-compose.dev.yml
  docker-compose.prod.yml
pyproject.toml
alembic.ini
README.md

```

Listing 9: Suggested top-level layout.

20 Appendix C — The Administrator’s One-Page Checklist

In plain language: Print this page. Bring it to every milestone demonstration. If every box is ticked, sign and proceed.

Milestone gate checklist

Milestone: _____ Date: _____

- ☐ Every behaviour listed in this RFP for this milestone has been demonstrated live.
- ☐ All automated tests passed today (green pipeline shown).
- ☐ Property-based tests for any rule-heavy code in this milestone are present and passing.
- ☐ Coverage thresholds (see §13) met or exceeded.
- ☐ At least one permitted user and one denied user demonstrated for each protected page.
- ☐ A known action was performed and the audit log entry was visible within seconds.
- ☐ Soft delete and hard delete both demonstrated; hard delete denied to a non-System-Admin user.
- ☐ Three random screens were checked at 360, 768, 1280, 1920 px and rendered correctly.
- ☐ The single theme renders consistently on every page demonstrated.
- ☐ The module’s documentation file (`docs/modules/<module>.md`) has been updated for this milestone.
- ☐ Security scans (`bandit`, `pip-audit`, `trivy`) are clean of high/critical findings.
- ☐ List views render under 1.5 s with the synthetic 5 000-row dataset.

Administrator’s signature: _____ Engineering lead’s

signature: _____

End of Specification — Version 3.0